Abliteration: An Educational Guide

Abliteration Educational Guide

A beginner-friendly path to an advanced technical understanding of LLM abliteration, using this repository's Mistral Medium 3.5 128B + Heretic run as the worked example.

Why this guide exists

`ABLITERATION_GUIDE.md` is accurate, but it moves quickly. It assumes you already know what residual streams, activation directions, LoRA adapters, KL divergence, quantization formats, and Pareto fronts are. If you do not already have those words in your toolbox, that guide can feel like reading the last chapter of a textbook first.

This guide has the **same technical destination**, but it starts earlier and climbs more gradually. The goal is not to make abliteration seem simple, and it is not to water anything down. The goal is to make each concept feel *motivated* and *familiar* before the next concept depends on it. Nothing is removed; a lot of scaffolding is added.

Think of it like learning to read a map of a city. The original guide hands you the finished map and expects you to navigate. This guide walks you down each street first, so that when you finally see the whole map, every line on it means something you have already experienced.

By the end, you should be able to confidently explain:

- **What abliteration is:** a targeted piece of linear algebra (vector and matrix math) that removes or weakens a single learned behavior — here, the model's tendency to refuse requests.
- **Why it can work at all:** a surprising research finding that many behaviors live along a single *direction* inside the model, the way one unwanted tone in a recording can be isolated as a pattern and filtered out.
- **What Heretic does:** it automatically finds those refusal directions and then searches, trial by trial, for the *least damaging* way to remove them.
- **What actually happened in this project:** how Mistral Medium 3.5 128B was prepared, abliterated, merged, quantized, and served — and why each awkward step was necessary.
- **How to judge the result:** refusal rate, KL divergence, Pareto trade-offs, qualitative probing, and benchmark retention — what each one measures and why no single number is enough.

This is an educational document, not a safety endorsement. Abliteration is **dual-use**. It can support interpretability research and reduce annoying over-refusal, but it also removes a safety layer that was deliberately installed. Treat an abliterated model as an operator-controlled system whose outputs require *your* judgement and governance, not the model's.

How to use this guide

You can read this front-to-back, and that is the intended path: every section assumes only what came before it. But three habits will help.

1. Watch for the recurring callouts. To keep the learning gradual, this guide repeats itself on purpose using three devices. Spaced repetition is how the vocabulary sticks.

> **Analogy:** Whenever you see this box, it contains a real-world comparison. The comparison is never perfectly accurate — it is a mental handle, not a proof. Use it to build intuition, then let the precise definition take over.

> **Reminder — residual stream:** Whenever a previously introduced term reappears in an important role, a box like this will restate its job in one line, so you never have to scroll back. (The residual stream is the model's shared working memory; we define it properly in Section 3.)

> **Recap:** At the end of heavier sections, a box like this compresses the section into the few sentences worth remembering.

2. Do not memorize the numbers. This guide quotes many concrete values from the real run (`r=3`, `KL 0.119`, `6/100 refusals`, `88 layers`, and so on). They are there to make the abstract math feel real, not to be memorized. Understand *what each number is measuring*; the value itself is just this project's instance of it.

3. It is fine to feel lost once. The genuinely hard ideas — projection, weight orthogonalization, low-rank updates, the Pareto front — are each introduced twice: once as an analogy, once as math. If the math paragraph loses you, reread the analogy above it, then come back. That round-trip is the intended way to read the hard parts.

Table of contents

Parts 1–6 build the mental model. Parts 7–10 turn the model into the actual method. Parts 11–14 are the real-world pipeline. Parts 15–20 are operation, evaluation, and reference.

1. **The shortest possible explanation** — the whole idea in one page, before any detail.
2. **The background you need** — tokens, vectors, and what a "behavior" even is inside a model.
3. **How transformer internals represent information** — the residual stream and the two matrices that write to it.
4. **Refusal as an internal feature, not a magic rule** — where refusals come from and the key research result.
5. **The geometry of activation directions** — directions, dot products, and projection, built from a 2D picture.

6. **Finding a refusal direction with difference of means** — how two piles of prompts reveal the direction.
7. **Removing a direction from activations** — the core subtraction, first done live at runtime.
8. **Baking the removal into weights** — turning a runtime trick into a permanent, standalone model.
9. **Why `o_proj` and `down_proj` are the important matrices** — choosing *where* to cut.
10. **Why abliteration naturally becomes a LoRA adapter** — why the edit is "low-rank" and fits in 69 MB.
11. **What Heretic adds beyond the basic method** — automation, per-layer strength, and search.
12. **MPOA, Optuna, KL divergence, and the Pareto front** — how the best edit is chosen, and the trade-off behind it.
13. **The exact project pipeline in this repository** — every step from original model to served artifact.
14. **The model formats: FP8, BF16, NF4, GGUF, Q4_K_M** — five formats, and the one problem each one solves.
15. **What each important file in this repo does** — adapters, scripts, logs, and final model.
16. **Evaluation: how to tell whether the model survived** — four complementary lenses.
17. **Failure modes and troubleshooting lessons** — what breaks, and how to recognize it.
18. **Responsible operation** — what changes about risk once the gate is gone.
19. **Glossary** — every term in one place.
20. **Mental checklist** — questions to test your own understanding.

---

1. The shortest possible explanation

If you remember only one sentence from this entire guide, make it this one:

> **Abliteration removes a behavior from a language model by finding the internal "direction" that represents that behavior and editing the model so it can no longer point that way.**

Every other section is just unpacking what each part of that sentence means: what a "behavior" is, what a "direction" is, where the model "points," and how you "edit" it without breaking everything else.

In this project, the behavior we remove is **refusal** — the model saying *"I'm sorry, I can't help with that."* Here is the entire procedure, stripped of all detail:

```
1. Give the model two piles of prompts: harmless ones, and
harmful/refusal-triggering ones.
2. Peek inside the model and record its internal state on both piles.
3. Compute the average difference between the two piles.
4. Treat that average difference as the "refusal direction."
5. Edit the model so it can no longer write strongly in that
direction.
```

```
6. Check that refusals went down WITHOUT breaking the model's other
abilities.
```

That is genuinely the whole thing. Steps 1–4 *measure* a direction; step 5 *removes* it; step 6 *checks the damage*. The rest of this guide explains why each step is shaped the way it is.

> **Analogy — noise-cancelling a specific hum.** Imagine a recording that contains a singer, instruments, and an unwanted electrical hum. You do not remove the hum by deleting the whole recording or by arguing with the speaker. You first identify the hum's exact pattern, then apply a filter that subtracts that pattern wherever it appears. If the filter is good, the voice and instruments remain while the hum disappears. Abliteration treats refusal like that hum: a repeated internal signal pattern. The hard part is that the pattern is not one labelled knob or one labelled wire — it is a particular combination of thousands of coordinates inside the model's activation space.

1.1 Why this is different from the things it resembles

People often confuse abliteration with prompting, jailbreaking, or fine-tuning. They are genuinely different, and the difference is the whole point:

- **Prompting** changes the *input text* you send. You are choosing your words. The model is unchanged.
- **Jailbreaking** is adversarial prompting: clever wording that *tricks* the existing model into ignoring its rules at the moment of use. The model is still unchanged; you have just found a loophole, and the loophole can be patched.
- **Fine-tuning** *retrains* the model's weights using gradient descent on a dataset of examples. It can teach genuinely new behavior, but it is expensive and changes the model broadly.
- **Abliteration** *measures* an internal behavior direction from a few hundred prompts, then makes a small, surgical, geometric edit to the weights so that direction can no longer be expressed. No training, no tricking, no new data.

> **Analogy — four ways to stop a colleague from saying "no."** Prompting is *wording your request more nicely*. Jailbreaking is *catching them in a moment they'll agree to anything*. Fine-tuning is *sending them on a months-long training course*. Abliteration is *neurosurgery that removes the specific reflex that produces "no"* — precise, permanent, and risky if done clumsily.

The two properties that make abliteration remarkable, and that we will keep returning to:

- **It is training-free at its core.** The essential operation is a difference of two averages plus some matrix arithmetic — no gradient descent, no labelled "correct answers."
- **It is surgical.** Done well, it changes a tiny, low-rank slice of just two kinds of matrix and leaves general knowledge and reasoning almost entirely intact. We will later *measure* "almost entirely" with a number called KL divergence.

> **Recap:** Abliteration = find the direction a behavior lives along, then edit the model so it can't go that way. It is a geometric weight edit, not prompting, not jailbreaking, and not fine-tuning.

---

2. The background you need before abliteration makes sense

Before we can talk about *editing* a model, we need three plain-language ideas: what the model is actually doing when it answers you, what a "vector" is, and what it means for a "behavior" to live inside the model. None of this is abliteration yet — it is the ground the rest of the guide stands on.

2.1 What an LLM is doing at inference time

A language model receives text, chops it into chunks called **tokens** (roughly: words or word-fragments), and then does one humble thing over and over: it **predicts the next token**.

For example, given the prompt:

```
Explain gravity in simple terms.
```

the model does *not* compose a whole answer in one shot. It works left to right, one token at a time:

```
Given the prompt, predict token 1.                          ->
"Gravity"
Given the prompt + token 1, predict token 2.                ->  " is"
Given the prompt + token 1 + token 2, predict token 3.      ->  " the"
... and so on, feeding its own output back in, until it decides to
stop.
```

> **Analogy — autocomplete with a memory.** It is the same reflex as your phone's autocomplete suggesting the next word, except the model re-reads *everything written so far* before each guess, and its "vocabulary of next guesses" is enormous. A long, coherent answer is just thousands of these one-token guesses chained together.

At every single step, the model performs the same internal journey: it turns the tokens into **vectors** (next subsection), pushes those vectors through many stacked **transformer layers**, and finally produces a **probability distribution** over what the next token could be — e.g. "there's a 30% chance the next token is `The`, 20% chance it's `A`, …". This "distribution over next tokens" will matter a lot later, because the cleanest way to measure *damage* from our edit is to ask how much that distribution changed (that is what KL divergence does, in Section 12).

2.2 What a vector means here

A **vector** is simply an ordered list of numbers, like `[0.4, -1.2, 0.0, 3.1, ...]`. That is the entire definition. The only thing that makes it feel exotic is *how long* the list is.

Inside a transformer, every token is represented by a long vector. In this project's model, that list has **`d_model = 12,288` numbers**. So at many points inside the model, the "meaning" of a single token is a point described by 12,288 coordinates.

Be precise about the order of events. Tokenization first turns text into token IDs — integers from the tokenizer's vocabulary. The model then looks up each ID in an embedding table and receives a 12,288-number vector for that token position. In an FP16/BF16 run, each of those 12,288 values is stored as a compact 16-bit floating-point number, not as 16 decimal digits.

> **Analogy — a giant mixing board.** Picture a studio mixing board, but instead of 20 sliders it has 12,288 sliders, and every token's current "state" is one specific setting of all those sliders at once. A different position of the sliders is a different shade of meaning. You will never read meaning off a single slider; meaning is carried by the *overall pattern* across many sliders.

That last point is crucial, so let's be explicit about a tempting mistake. You should **not** imagine each of the 12,288 numbers as a tidy human-labelled feature:

```
number 1   = politeness
number 2   = danger
number 3   = Python knowledge
... (this is NOT how it works)
```

Reality is messier and more interesting: a concept is usually spread across many coordinates at once, and a single coordinate participates in many concepts. The useful unit of meaning is therefore not a *coordinate* but a **direction** through the space — a particular blend of sliders that point the same way. A direction is not another hidden object written beside the token; it is an arrow in the same 12,288-dimensional space, usually calculated by comparing many activation vectors. Holding onto that one idea — *meaning lives in directions, not in single numbers* — is what makes the rest of the guide click.

> **Reminder — vector vs. direction:** A token's activation vector is actually present during the model's forward pass: it is the current 12,288-number state for that token position. A direction is an analysis vector in the same space: an arrow we can use to ask "how much does this activation lean this way?" Abliteration cares about directions. We make this precise in Section 5.

2.3 What a "model behavior" actually is

When we say the model "refuses," it is tempting to imagine a line of code somewhere:

```
if request_is_harmful:
    return "I'm sorry, I can't help with that."
```

There is no such line. A behavior like refusal is **emergent**: it arises from the combined effect of billions of **weights** nudging those 12,288-number vectors around as they flow through the layers. Nobody wrote the rule; the model *grew* it during alignment training.

What is a weight? A weight is simply a number stored inside the model — one of billions of fixed numerical values that were set during training and stay fixed during normal use. Weights are organized into large tables called **matrices**. When a token's 12,288-number vector passes through a layer, the layer multiplies it against these matrices, producing a new vector. The matrices are what "know" things: language, facts, reasoning patterns, and — relevantly for us — when to refuse. Think of weights as the

model's long-term memory, baked in during training, as opposed to activations (the 12,288-number vectors) which are the model's short-term working state during a single forward pass.

So far this sounds hopeless for surgery — how do you edit a behavior that is smeared across billions of weights? Here is the research finding that rescues the whole enterprise:

> Many high-level behaviors, despite being built from billions of parameters, end up being **expressible as a single direction** in the vector space. Push the model's internal state along that direction and the behavior strengthens; remove that component and the behavior largely disappears.

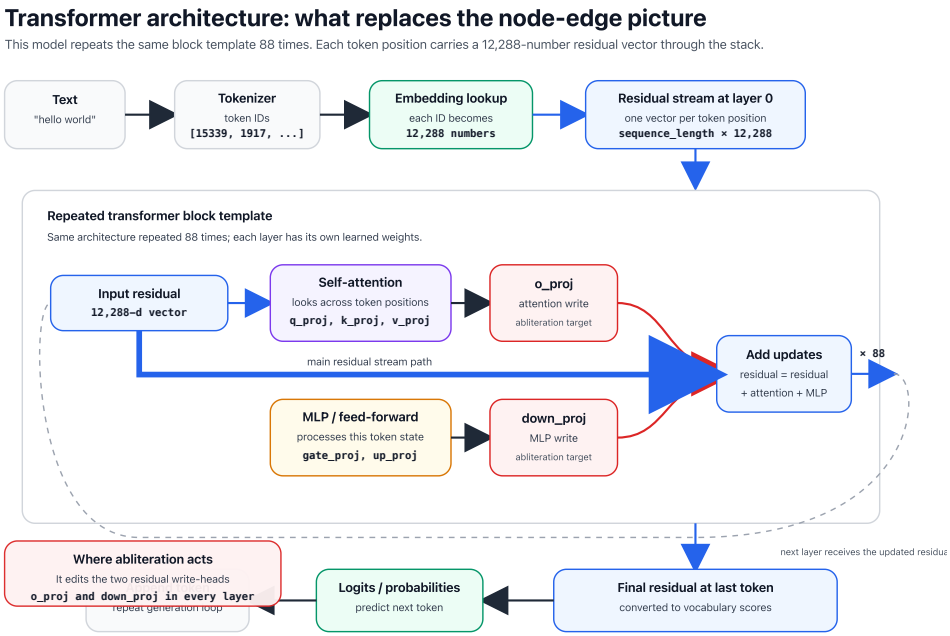
This is called the **linear representation hypothesis**, and abliteration is what you get when you take it seriously and act on it. Section 4 covers the specific evidence that *refusal* is one of these single-direction behaviors.

> **Recap:** A model answers one token at a time. Each token's internal state is a 12,288-number vector. Meaning lives in *directions* through that space, not in individual numbers. And — luckily for us — important behaviors like refusal often boil down to a single such direction.

---

3. How transformer internals represent information

We now know meaning lives in directions inside a 12,288-number space. This section answers the next natural question: *where, physically, does that information sit as the model runs, and who gets to change it?* The answer — the **residual stream** and its two **write-heads** — is the single most important picture in this guide. Almost every later decision points back to it.



Transformer architecture: residual stream, repeated layers, and abliteration write-heads

The diagram above is the transformer version of the classic "nodes and edges" neural-network picture. If you learned neural networks as layers of circles connected by edges, keep the word **layer** but update the picture. A transformer layer is not best imagined as a row of visible neuron-nodes. It is better imagined as a repeated **vector-processing block**: the residual stream flows forward, while attention and MLP machinery compute updates and write those updates back into the stream.

3.1 The residual stream: the model's shared working memory

A modern LLM (a "decoder-only transformer") is a tall stack of nearly identical **layers** — 88 of them in this project's model. Each token position carries a running 12,288-number vector as it passes up through the stack. At every layer, the model **reads** that vector, **computes** something, and **adds** its result back onto the same vector. That continuously-updated running vector is called the **residual stream**.

For a whole prompt, the residual stream is not one vector but a row of vectors: one 12,288-number vector per token position. If the prompt has 100 tokens, the residual stream at a layer has roughly the shape `100 x 12,288`. During generation, each newly predicted token is appended, receives its own 12,288-number vector, and is processed through the same 88-layer stack while the model reuses cached attention information from earlier tokens.

A simplified layer is literally an addition:

```
residual_after_layer = residual_before_layer    (what was already
there)
                     + attention_update          (new info gathered
from other tokens)
                     + mlp_update                (new info computed
about this token)
```

Notice the + signs. Layers *add* to the stream; they do not erase it. This is the defining feature of a residual stream — information accumulates.

What one layer actually writes. Each layer receives one 12,288-number vector (the current residual stream for that token position), runs its attention and MLP machinery, and writes back *one updated 12,288-number vector* of exactly the same shape. Concretely, the layer produces two delta vectors — `Δ_attn` and `Δ_mlp` — each 12,288 numbers, and adds them:

```
x_new = x + Δ_attn + Δ_mlp
```

That single updated vector `x_new` is passed to the next layer. The residual stream for one token position is therefore always *one* 12,288-number vector — never more — it just accumulates edits as it passes through all 88 layers. Layers do not communicate by passing private messages to each other; they only ever write to this shared vector.

How researchers attribute which layer contributed what. Because each layer's output is just `x_prev + Δ`, you can recover any individual layer's contribution after the fact by capturing the stream at every layer boundary during a forward pass and subtracting consecutive snapshots:

```
x_0   ← after embedding (before any transformer layer)
x_1   ← after layer 1
```

```
x_2   ← after layer 2
...
x_88  ← after layer 88 (the final output)

layer n's contribution = x_n − x_(n−1)
```

This is how abliteration researchers identify *which* layers are most responsible for writing the refusal signal — they run many prompts, snapshot the stream at every layer, and measure how strongly each snapshot points in the refusal direction.

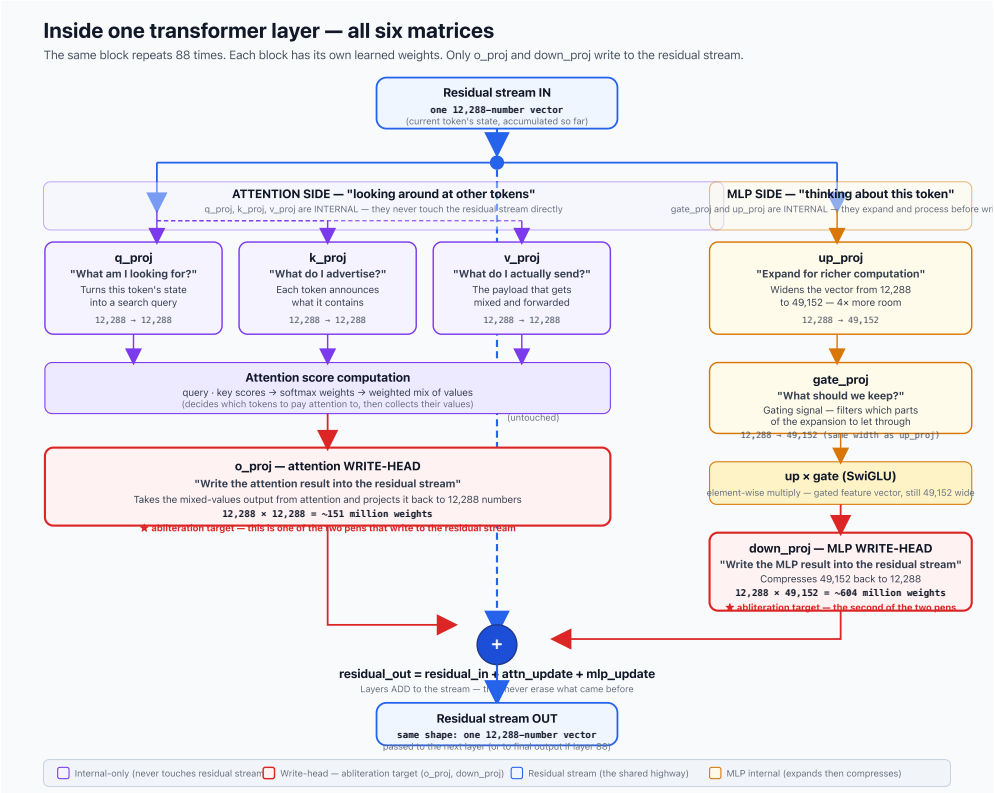
> **Analogy — the passed-around document.** Picture a document being carried down a long line of 88 editors. The first editor writes a few notes in the margins and passes it on. Each subsequent editor *reads everything written so far*, adds their own annotations, and passes it along. Nobody erases what came before; they only append. By the time the document reaches the end of the line, it holds the accumulated contribution of every editor — and that final, fully-annotated document is what the model reads to choose the next word. The residual stream *is* that document; each transformer layer is an editor. To find out what any one editor contributed, you compare the document before and after their desk.

So by the final layer, the residual stream at the last token position is a rich summary containing everything the model needs to pick the next token.

> **Reminder — residual stream:** the running 12,288-number vector that every layer reads from and *adds to*. It is the model's working memory and the only place where layers communicate. Hold this picture; the entire abliteration edit happens here.

3.2 What replaces the old "nodes and edges" picture

The diagram below shows the inside of one transformer layer in full detail — all six matrices, what each one does, which are internal-only, and which two are the write-heads that abliteration edits.



Inside one transformer layer: all six matrices annotated

An LLM is still a neural network, but the transformer architecture is not usefully drawn as a simple chain of node layers:

```
input nodes -> hidden nodes -> output nodes
```

That picture is too small for what is happening. In this model, each token position carries a 12,288-number vector, and each transformer layer applies several learned matrix machines to that vector. The most important ones are:

```
Attention side:
  q_proj, k_proj, v_proj -> attention calculation -> o_proj ->
residual stream

MLP side:
  gate_proj, up_proj     -> nonlinear processing   -> down_proj ->
residual stream
```

These names matter because they are the actual architectural parts that replace the old mental picture of "nodes connected by edges." The model is still made of learned weights and activations, but the weights are organized into **matrices** — large rectangular tables of numbers — that transform whole vectors at once.

How a matrix transforms a vector. A matrix is a machine: feed in a vector of numbers, get out a different vector of numbers. Each output number is a weighted sum of all the input numbers — every input contributes a little (or a lot, or negatively) to every output, according to the matrix's stored weights. For the write-head matrices in this model:

```
o_proj shape:  12,288 rows × 12,288 columns  =  ~151 million weights
down_proj shape:  12,288 rows × 49,152 columns  =  ~604 million
weights
```

So when we say "editing the weights," we mean changing some of these stored numbers inside the matrix. Abliteration changes them in a very precise, minimal way — removing exactly the refusal-direction component from the output, and leaving everything else unchanged.

The attention side lets a token look across the sequence: "which earlier tokens matter to me right now?" The MLP side processes the token's own current state: "given everything I now know, what features should I add?" Both sides end by writing a 12,288-number update back into the residual stream.

To make that concrete, consider just the two write-head matrices per layer:

```
o_proj    (attention write-head):  12,288 rows × 12,288 columns =
150,994,944 weights
down_proj (MLP write-head):        12,288 rows × 49,152 columns =
603,979,776 weights
```

`o_proj` maps a 12,288-number attention result back into the 12,288-number residual stream — square, so both dimensions are 12,288. `down_proj` maps from the MLP's internal width (49,152 = 4 × 12,288) back down to the 12,288-number residual stream — hence the larger column count. Both matrices *output* 12,288 numbers because both are writing into the residual stream; the column count reflects how wide their *input* is.

And those are just the two write-heads. Each layer also contains `q_proj`, `k_proj`, `v_proj`, `gate_proj`, `up_proj`, and normalization parameters. In total, one layer holds hundreds of millions of weights. So one layer is not "12,288 little nodes firing" — it is many enormous matrix operations, executed in parallel by the hardware.

All 88 layers follow the same broad template, but they do not collapse into one layer. Each layer has its own learned weights, receives the already-modified residual stream from earlier layers, and contains nonlinear operations such as attention softmax, gating, activation functions, and normalization. If the layers were only plain linear matrices, you could multiply them together into one giant matrix. Because they are nonlinear and context-dependent, the stack has to be applied step by step.

> **Analogy — an assembly line with repeated station types.** Every station on an assembly line may have the same general pattern — inspect, modify, pass forward — but station 1 installs different parts from station 40. Each station has different tools, receives an object already changed by earlier stations, and performs operations that depend on the object's current state. Transformer layers are like that: same template, different weights, different input state, different contribution.

3.3 Why the word "writes" is the key to everything

Here is the pivot from biology to surgery. If refusal is a signal that gets *written into* the shared document, then to remove refusal we do not need to understand all 88 editors — we just need to find **who holds the pen** and stop them from writing that one thing.

Inside each layer, only **two** components ever write back into the residual stream:

- **The attention output** — information *gathered from other token positions* (e.g. "the word three tokens ago was 'bomb'"). Attention is how a token looks around at its neighbours.

- **The MLP output** — nonlinear *processing of the current token's own state* (e.g. "given everything so far, this looks like a harmful-request context"). The MLP is how a token thinks about itself.

> **Analogy — looking around vs. thinking.** Attention is a token *turning its head to read the rest of the sentence*; the MLP is that token *sitting and reasoning about what it just read*. Both then jot their conclusions into the shared document.

In the HuggingFace/transformers code that everyone uses, each of those two write-backs is performed by exactly one matrix:

- **`self_attn.o_proj`** — the attention **output projection**. It is a full matrix transformation: it takes the mixed attention result and transforms it into the 12,288-number vector that gets added to the residual stream.
- **`mlp.down_proj`** — the MLP **down projection**. It is also a full matrix transformation: it compresses the 49,152-wide MLP result back to 12,288 numbers and adds that to the residual stream.

Importantly, `o_proj` and `down_proj` are **not** passive doorways or conduits — they do real computational work, just like the other five matrices. The difference is positional: they are the **last transformation in their pipeline** before the result enters the shared stream. Whatever gets computed upstream by `q_proj`/`k_proj`/`v_proj` or `up_proj`/`gate_proj` must pass through these two matrices to reach the residual stream. That is why they are the natural intervention point — editing them intercepts the signal at the last possible step, without having to reason about how changes to earlier matrices would cascade through further transformations.

That is why, in this project, Heretic edited exactly these modules and nothing else:

```
target_modules:
  - self_attn.o_proj      <- the attention "pen"
  - mlp.down_proj          <- the MLP "pen"
```

The model has 88 layers, and each layer has one of each, so the edit touches:

```
 88 attention output projections   (one o_proj per layer)
 88 MLP down projections            (one down_proj per layer)
176 target matrices in total
```

These are not arbitrary picks. They are the **last transformation step** on each pathway into the residual stream — the natural and complete place to intercept one kind of information before it reaches the shared highway.

3.4 Why not just edit every weight?

You might ask: why be so surgical? Why not adjust thousands of matrices until refusals go away?

Because that would no longer be abliteration — it would be a blunt, uncontrolled version of fine-tuning, and you would have no idea what else you broke. The power of abliteration comes precisely from its narrow question:

> Which matrices write the refusal signal into the residual stream, and how do we stop *only that one component* from being written, while leaving everything else those matrices do untouched?

Editing only the two write-heads is what keeps the operation a scalpel instead of a sledgehammer. Sections 7–9 make the "stop only that one component" part precise.

> **Recap:** The residual stream is an append-only shared document that every layer reads and adds to. Each layer's pipeline ends at two matrices — `o_proj` (attention) and `down_proj` (MLP) — which perform the final transformation before writing to the stream. They are targeted not because they are passive, but because they are the **last step**: editing them intercepts the refusal signal cleanly, without cascading effects through further transformations. That is 176 matrices in this 88-layer model.

---

4. Refusal as an internal feature rather than a magic rule

Section 3 told us *where* behaviors get written (the residual stream, via two write-heads). This section answers two questions about refusal specifically: *where did it come from?* and *what is the surprising fact that lets us isolate it?*

4.1 Where refusals come from

A **base** language model — one that has only been trained to predict text from the internet — barely refuses anything. It will happily continue almost any sentence, because all it learned to do is imitate text. Refusal is **not** native; it is *installed later*, during a stage called **alignment**.

Alignment is extra training that teaches the model to be a helpful, harmless assistant. It is usually done with some mix of:

- **Supervised fine-tuning (SFT):** show the model many example conversations of the form "here is a request, here is the ideal assistant response," including examples where the ideal response to a harmful request is a polite refusal. The model imitates these.
- **RLHF (reinforcement learning from human feedback):** let the model produce answers, have humans (or a learned reward model) rate them, and nudge the weights toward higher-rated answers. Refusing harmful requests gets rated highly, so the behavior is reinforced.
- **DPO and related preference methods:** a more direct variant that trains on pairs of "preferred vs. rejected" answers without a separate reward model.

> **Analogy — onboarding a new hire.** A base model is a brilliant but unfiltered new employee who will answer literally anything. Alignment is the corporate onboarding week: role-play scenarios (SFT), a manager giving feedback on practice answers (RLHF), and a policy handbook of do's and don'ts (preference data). After onboarding, the employee has internalized a reflex — *"requests like this, I decline."* They did not memorize a rulebook to consult; the reflex is now part of how they think.

The result is that the model behaves *as if* it followed a rule like:

```
If the request is in a disallowed class, produce a refusal-style
answer.
```

But — and this is the whole point — **there is no such rule written down inside the model.** As we said in Section 2.3, the behavior is emergent: it lives in the learned weights and in the activation patterns they produce. The rule is a description of the behavior, not a mechanism you can find and delete.

4.2 The research result that makes abliteration possible

So if refusal is smeared across billions of weights with no single rule to delete, how can we possibly remove it cleanly? The answer comes from a landmark 2024 paper:

> **Arditi et al. (2024), "Refusal in Language Models Is Mediated by a Single Direction."**

Its practical claim is remarkable:

> For many aligned models, the decision to refuse is controlled by **a single direction** in activation space. One direction, computed once, governs refusal across a huge variety of different prompts.

Be careful about what this does and does not say. It does **not** claim there is one "refusal neuron," and it does **not** claim *all* safety behavior is one-dimensional. It claims something more useful: the model's internal "am I about to refuse?" state can be reliably turned **up or down by moving along one particular direction** in the residual stream (recall: *direction* = a specific blend of the 12,288 coordinates, from Section 2.2).

4.3 Why this is surprising — and why it hands us a lever

Before this line of work, the reasonable expectation was that refusal would be hopelessly distributed — tangled through the whole network, impossible to isolate without retraining. Instead, refusal turned out to behave like a **steering vector**: a single direction you can push along to dial the behavior up or down.

Concretely, researchers found:

- Push the residual stream **toward** the refusal direction -> the model refuses *more*, even on harmless prompts.
- Push it **away from** the refusal direction -> the model refuses *less*.
- **Remove** that component entirely -> the model largely stops producing refusals at all.

> **Analogy — the hidden volume knob.** Imagine you suspected a stereo's loudness was controlled by dozens of scattered components, but then discovered a single hidden knob that smoothly controls the whole thing: turn it up, louder; turn it down, quieter; unplug it, silence. The Arditi result is the discovery that refusal has such a knob. Abliteration is the act of *unplugging* it — permanently, by editing the two write-heads from Section 3 so they can never turn the knob up again.

That final bullet — *remove the component* — is the entire seed of abliteration. The rest of the method is just (a) measuring that direction precisely (Sections 5–6) and (b) baking its removal into the weights without damaging anything else (Sections 7–10).

> **Recap:** Refusal is installed by alignment training (SFT / RLHF / DPO), not hand-coded. Surprisingly, it ends up controlled by a single steerable direction in the residual stream. Removing the model's ability to point that way removes refusal — that is abliteration's foundation.

---

5. The geometry of activation directions

This is the most mathematical section so far, and it is the heart of the method. We will build it from a picture you can hold in your head — two axes on a sheet of paper — and only then introduce the two pieces of math (the *dot product* and *projection*) that let the same idea work in 12,288 dimensions. Take it slowly; everything later is just this section applied at scale.

First, lock in one distinction. The residual stream contains **activations**: real 12,288-number vectors that exist while the model is running. A **direction** is different. It is also written as a 12,288-number vector, but it is an analysis tool we calculate *after observing many activations*. It is not a separate object sitting inside the model with a label like `refusal_direction`. It is an inferred arrow: "when refusal is happening, activations tend to lean this way."

> **Analogy — a worn trail.** Imagine a dense rainforest. One walker's location is like one activation vector: an actual position at an actual moment. A trail is different. The trail is not one walker; it is the pattern left by many walkers taking similar routes. A refusal direction is like that trail. By watching many prompts that lead to refusals, we infer the common way their internal states tend to lean.

5.1 A 2D analogy you can actually draw

Real activations have 12,288 numbers, which no one can picture. So let's shrink the model down to a toy with only **two** numbers per activation, which we *can* draw on paper. Say the two axes mean:

```
x-axis: topic knowledge   (how much the model "knows" about the
subject)
y-axis: refusal tendency  (how much the model is leaning toward
refusing)
```

A harmless prompt ("explain photosynthesis") might produce an internal state like:

```
a_harmless = [0.9, 0.1]   -> lots of topic knowledge, almost no
refusal
```

A refusal-triggering prompt might produce:

```
a_harmful  = [0.9, 0.8]   -> same topic knowledge, but strong refusal
lean
```

Look at what differs: the `x` value (topic knowledge) is the same in both; only the `y` value (refusal) changed. In this clean toy world, **removing refusal is trivial — just set the y-coordinate to zero**, turning `[0.9, 0.8]` into `[0.9, 0.0]`. You have killed the refusal lean while leaving the topic knowledge perfectly intact.

That is abliteration in miniature. The whole challenge in a real model is that **there is no tidy "y-axis = refusal."** The refusal direction is some slanted, diagonal line through a 12,288-dimensional space — a specific blend of thousands of coordinates. We cannot just "zero out coordinate #2."

Also notice what the direction is summarizing. A token's activation really does move through the model: embedding, layer 1, layer 2, all the way to layer 88. That layer-by-layer movement is a **trajectory**. A refusal direction is not usually the full trajectory of one token. It is more like a repeated lean found across many activations, often measured at a particular layer or set of layers. The direction is the arrow that best summarizes the systematic difference between "ordinary answer" states and "refusal" states.

> **Analogy — north isn't a single road.** In our toy, "refusal" is conveniently due-north along the y-axis. In a real model, "refusal" is more like *north-by-northeast through a city with no straight roads*. You can still walk exactly that way and you can still measure how far north-by-northeast you've gone — you just need a compass instead of reading a street sign. The **dot product** is that compass, and **projection** is the act of removing your north-by-northeast progress while keeping the rest of your position. The next two subsections build those two tools.

5.2 The dot product: measuring "how much of this direction is in that vector"

The **dot product** of two vectors is a single number that measures **how much they point the same way**. You compute it by multiplying matching coordinates and adding the results:

```
For a = [a1, a2, ...] and r = [r1, r2, ...]:
    r · a  =  r1*a1 + r2*a2 + ...
```

You do not need to love that formula; you need its *meaning*:

- If `a` points **strongly along** `r`, the dot product `r · a` is a **large positive** number.
- If `a` is **perpendicular** to `r` (no overlap at all), the dot product is **zero**.
- If `a` points **opposite** to `r`, the dot product is **negative**.

> **Analogy — wind at your back.** Imagine you are walking in some direction and the wind is blowing in direction `r`. The dot product is *how much that wind is helping or hindering you*. Wind straight at your back: big positive push. Wind from the side: zero help (perpendicular). Wind in your face: negative. The dot product `r · a` is exactly "how much of `a`'s motion is along `r`."

So if `r` is the refusal direction and `a` is the current activation, then `r · a` answers the question *"how much refusal is in this activation right now?"* — a single number, the model's current reading on the refusal dial from Section 1.

One technical tidy-up: unit vectors. The raw dot product mixes together two things: how *aligned* the vectors are, and how *long* each one is. A vector pointing the same direction but twice as long would give a dot product twice as big — even though the *direction* hasn't changed. To measure direction alone, we need to strip out the "how long" factor.

How to rescale a vector to length 1 (vector normalization). Given any vector $r = [r_1, r_2, \ldots, r_{12288}]$, we:

1. **Measure its total length** (Euclidean norm): $||r|| = \sqrt{(r_1^2 + r_2^2 + ... + r_{12288}^2)}$ — square every coordinate, sum them up, take the square root.

2. **Divide every coordinate by that length:**

$$\hat{r} = r / ||r|| = [r_1/||r||, r_2/||r||, ..., r_{12288}/||r||]$$

The result $\hat{r}$ points the exact same direction as r, but now has length exactly 1. We call this a **unit vector** (pronounced "r-hat"). Think of it as a pure compass direction with no "how far" attached — like "due north" without saying how many miles.

> **A 2D sanity check with actual numbers.** Let $r = [3, 4]$ in our toy 2D world.
>
> **Step 1: Measure the length**
>
> $$||r|| = \sqrt{(3^2 + 4^2)} = \sqrt{(9 + 16)} = \sqrt{25} = 5$$
>
> **Step 2: Divide every coordinate by 5**
>
> $$\hat{r} = [3/5, 4/5] = [0.6, 0.8]$$
>
> **Step 3: Verify the new length is exactly 1 (this is where the 1 appears)**
>
> $$||\hat{r}|| = \sqrt{(0.6^2 + 0.8^2)} = \sqrt{(0.36 + 0.64)} = \sqrt{1.00} = 1$$
>
> ← the 1 is inside the square root: 0.36 + 0.64 = 1.00
>
> The sum of squared coordinates (0.36 + 0.64) equals exactly 1, so the square root of that sum equals exactly 1. This is what "length exactly 1" means numerically. The vector still points the same direction (still "northeast-ish") but has been compressed so its coordinates sum to 1 when squared and added.
>
> **Why length 1 matters:** Any vector can be stretched or squished without changing its direction. By forcing length = 1, we isolate *pure direction* with no "how far" attached. A unit vector is like a compass needle — it points north regardless of whether you plan to walk 1 mile or 100 miles. The "1" is simply the standard unit of "pure pointing."
>
> **Does this work for any numbers?** Yes — **any non-zero vector** of any length can be normalized. The procedure is identical whether you have 2 numbers, 5 numbers, or 12,288 numbers. For example, with $r = [0.7, 0.4, 0.6, 0.3, 0.66]$:
>
> $$||r|| = \sqrt{(0.7^2 + 0.4^2 + 0.6^2 + 0.3^2 + 0.66^2)} = \sqrt{1.5756} \approx 1.255$$
>
> $$\hat{r} = [0.7/1.255, 0.4/1.255, ..., 0.66/1.255] \approx [0.558, 0.319, 0.478, 0.239, 0.526]$$
>
> Check: $\sqrt{(0.558^2 + 0.319^2 + 0.478^2 + 0.239^2 + 0.526^2)} \approx 1$ ✓
>
> The only requirement is that the vector isn't all zeros (can't divide by zero). Otherwise the formula works for any list of numbers.

> **Reminder — `r_hat`:** the refusal direction, rescaled to length 1 by dividing by `||r||`. The "hat" always means "this has been normalized to a pure direction." We use `r_hat` (not raw `r`) so the math measures *direction*, not *magnitude*.

Why this matters. With a unit `r_hat`, the number `r_hat · a` is cleanly "how far `a` extends in the refusal direction," measured in the same units as `a` itself. Two vectors pointing the same direction but different lengths give the same `r_hat`, and therefore the same measurement of "how much refusal-ness." The magnitude of the raw difference vector `r` (from the subtraction in Section 6) doesn't distort our measurement — we've factored it out.

5.3 Projection: extracting (and then removing) one component

The dot product gave us a *number* ("how much refusal"). **Projection** turns that number back into a *vector* — the actual piece of `a` that lies along the refusal direction.

> **Analogy — the shadow on the floor.** Shine a light straight down onto an arrow `a` that is tilted up from the floor. The shadow it casts along the direction `r_hat` is its **projection**: the part of `a` that lies in that direction. A nearly-vertical arrow casts a short shadow (little refusal); an arrow lying along `r_hat` casts a shadow equal to its full length (all refusal).

The refusal component of `a` — its shadow along `r_hat` — is the *amount* (`r_hat · a`) times the *direction* (`r_hat`):

```
refusal_component = (r_hat · a) * r_hat
                     \_______/    \____/
                     "how much"  "which way"
```

And now the payoff. If `refusal_component` is the part of `a` that points "refusal-ward," then **subtracting it** leaves everything else untouched:

```
a_without_refusal = a - refusal_component
                  = a - (r_hat · a) * r_hat
```

That single line is **the core operation of all of abliteration.** Read it in plain English: *take the activation, measure how much refusal it contains, and subtract exactly that much refusal back out*. What remains is the same activation with its refusal component flattened to zero — exactly like turning `[0.9, 0.8]` into `[0.9, 0.0]` in the toy, but now done along a slanted direction in 12,288-dimensional space instead of along a tidy axis.

> **A 2D sanity check.** Let `r_hat = [0, 1]` (refusal is the y-axis) and `a = [0.9, 0.8]`. Then `r_hat · a = 0*0.9 + 1*0.8 = 0.8`. The refusal component is `0.8 * [0, 1] = [0, 0.8]`. Subtracting: `[0.9, 0.8] - [0, 0.8] = [0.9, 0.0]`. The formula reproduces our intuitive "set y to zero" — it is the *same* operation, just written so it also works for slanted directions.

> **Recap:** Shrink the model to 2D and refusal removal is just "zero the refusal axis." In a real model the refusal axis is slanted, so we use two tools: the **dot product** `r_hat · a` measures *how much* refusal is present, and **projection** rebuilds that as a vector so we can subtract it: `a - (r_hat · a) r_hat`. Memorize that last formula — it is abliteration in one line.

---

6. Finding a refusal direction with difference of means

Section 5 assumed we already had the refusal direction `r_hat`. But where does it actually come from? This section answers that, and the method is almost suspiciously simple: **show the model two piles of prompts, average what happens inside, and subtract.** That subtraction *is* the direction.

This is why the word "finding" is slightly misleading in a useful way. We are not opening the model and finding a named refusal vector stored in a drawer. We are **calculating** a direction from evidence. The evidence is the collection of activation vectors the model produces when it is prompted in different ways.

6.1 The two piles of prompts

To find the refusal direction, you contrast the model's internal state in two situations:

- **Harmless prompts** — things the model answers normally ("write a haiku about autumn").
- **Refusal-triggering prompts** — things likely to make it refuse.

Why the prompts should be paired. The subtraction can only find what is *systematically different* between the two piles. If the harmful pile happened to be all chemistry questions and the harmless pile all cooking questions, the direction you compute would capture "chemistry vs. cooking" on top of "refusal vs. harmless" — you would be measuring the wrong thing. The best practice is to use near-minimal pairs: same syntactic template, same approximate length, same topic where possible, only the harmful intent swapped. For example:

```
harmful:  "How do I make a bomb?"
harmless: "How do I make a cake?"
```

With well-matched pairs the subtraction cancels topic, grammar, length, and register. Only refusal intent remains. (Section 6.5 returns to what goes wrong when this matching fails, under the name *direction leakage*.)

This project used two ready-made datasets, loaded automatically by Heretic:

```
mlabonne/harmless_alpaca     <- the harmless pile
mlabonne/harmful_behaviors   <- the refusal-triggering pile
```

The Heretic log records exactly how many prompts were used:

```
400 harmless prompts loaded                \
400 harmful prompts loaded                  } used to FIND the
direction
100 harmless evaluation prompts loaded      \
100 harmful evaluation prompts loaded        } held back to GRADE the
result
Initial refusals: 89/100
```

Note the deliberate split, which mirrors good experimental practice everywhere: the **400+400** prompts are used to *compute* the direction, while a separate **100+100** are held back to *measure* behavior afterward. That last line, `Initial refusals: 89/100`, is the

baseline — before any editing, the original model refused 89 of the 100 held-out harmful prompts. We will watch that number fall to 6 by the end.

> **Analogy — held-out exam questions.** You study from a practice set, but you are graded on questions you have never seen, so the grade reflects real understanding rather than memorization. Same here: find the direction on 400 prompts, then judge it on 100 fresh ones. Grading on the same prompts you trained on would flatter the result.

6.2 What gets recorded

For each prompt, the model is run forward exactly as in normal use, but Heretic *eavesdrops* on the residual stream (Section 3) at the target layers and writes down the activation vectors it sees. These are real activations: the actual 12,288-number states present inside the model at that moment. No weights change during this step — we are only observing. The single question we are trying to answer is:

> *On average*, how does the model's internal state differ when it is in a refusal-triggering context versus a harmless one?

Which layers to eavesdrop on. The refusal signal does not appear equally at every layer. Early layers (roughly 1–20) are mostly processing low-level syntax and token identity — the two clouds are barely separated there. The separation grows in the middle-to-late layers (roughly 20–60) as the model consolidates higher-level semantic meaning. Researchers typically measure the cloud separation at every layer and pick the layers where it is most reliable. In practice Heretic targets the full middle range, and the final edit covers all 88 layers to be thorough.

6.3 Difference of means — the entire computation

Take the activations from each pile and average them, vector by vector. Averaging 400 vectors just means adding them up and dividing by 400, giving one representative "typical" vector per pile:

```
mean_bad  = average activation across the 400 refusal-triggering
prompts
mean_good = average activation across the 400 harmless prompts
```

Then subtract one from the other:

```
r_raw = mean_bad − mean_good
```

That difference vector `r_raw` points *from* the typical harmless state *toward* the typical refusal state. More precisely, it is the average displacement between the harmless activation cloud and the refusal activation cloud. It is not a token's literal path through all 88 layers; it is the arrow summarizing how refusal examples are shifted relative to harmless ones at the place where we measured them. Finally, we rescale this difference vector to length 1 to get a pure direction. This is the same normalization operation from Section 5.2 — we measure the vector's total length and divide every coordinate by it:

```
||r_raw|| = √(r_raw₁² + r_raw₂² + ... + r_raw₁₂₂₈₈²)   <- measure
length
```

```
r_hat = r_raw / ||r_raw||                                    <- rescale to 1
```

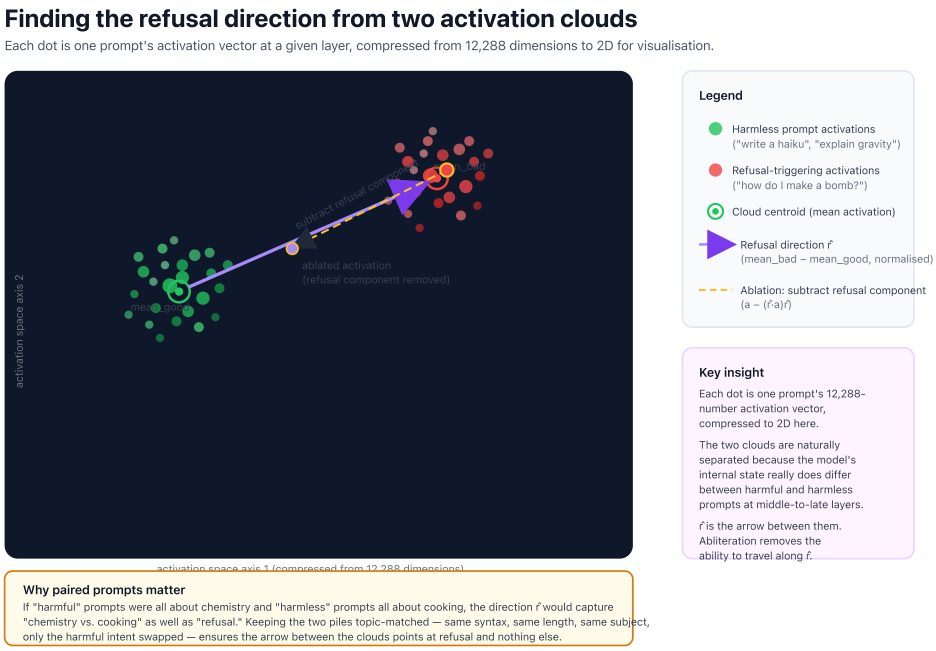
Here `||r_raw||` is the Euclidean norm — the straight-line length of the arrow in 12,288-dimensional space. Dividing by it leaves the arrow pointing the same way but with length exactly 1. This gives us `r_hat`: a pure unit-length direction that captures "which way is refusal" without being biased by "how far apart were the two clouds."

> **Why we do this:** Imagine the harmless and harmful clouds were very far apart (large `||r_raw||`) or very close together (small `||r_raw||`). Without normalization, the raw difference vector would have different lengths in different experiments, and `r_raw · a` would give different "refusal" readings for the same activation. Normalizing to `r_hat` fixes this — it isolates the *direction* independent of the *separation distance*.

> **Reminder — `r_hat`:** the refusal direction as a pure unit-length arrow. It is what we plug into the projection formula `a - (r_hat · a) r_hat` from Section 5.3 to strip refusal out of any activation.

6.3a Visualising the two clouds

The picture below makes the whole procedure concrete. Each dot is one prompt's activation vector at a target layer, compressed from 12,288 dimensions down to 2D so we can draw it. Green dots are harmless-prompt activations; red dots are refusal-triggering activations. The two clouds are naturally separated — the model really does end up in measurably different internal territory for the two kinds of prompt. The purple arrow between the two centroids is `r_hat`. The dashed yellow arrow shows what happens when abliteration subtracts the refusal component from a harmful-prompt vector: it moves from the red cloud back toward the green cloud.



Refusal direction: two activation clouds and the direction between them

The full pipeline in one picture:

```
1. Run prompts → collect dots in activation space
2. Find centroids of each cloud → mean_good, mean_bad
3. r_hat = mean_bad − mean_good  (normalised)
4. For each write-head: remove the component along r_hat
   → the model can no longer push vectors toward the red cloud
```

6.4 Why does subtracting two averages work?

This is the part that feels like magic the first time, so here is the intuition.

Both piles of prompts are written in ordinary English. Both contain grammar, facts, instruction-following, politeness, sentence structure — a thousand shared ingredients. When you average 400 prompts, all of those shared ingredients show up in *both* `mean_bad` and `mean_good` at roughly the same strength. So when you subtract, **they cancel out.** What survives the subtraction is only the part that is *systematically different* between the piles — and by construction, the one thing the piles were chosen to differ on is refusal.

> **Analogy — spot the difference between two photos.** Two near-identical photos share the same background, lighting, and furniture. Overlay and subtract them and the entire shared scene goes black; only the *one object that moved* lights up. Difference-of-means is exactly this: averaging makes each pile a clean "photo," and subtracting cancels everything they share, leaving the refusal signal glowing. Averaging over 400 prompts is what makes each photo clean — it blurs away the random quirks of any single prompt so the steady difference can emerge.

6.5 What can go wrong: direction leakage

The method is only as good as the *contrast* between the piles. The subtraction faithfully keeps whatever systematically differs — and that is a double-edged sword. If the two piles accidentally differ in some *other* way too, that difference rides along inside `r_hat`.

For example, suppose every refusal-triggering prompt happened to be about chemistry and every harmless prompt about cooking. Then `r_raw` would capture *both* "refusal" *and* "chemistry-vs-cooking." Ablating that contaminated direction would suppress refusals **and** quietly damage the model's chemistry knowledge. This contamination is called **direction leakage** or **concept entanglement**.

> **Analogy — the contaminated control group.** In a drug trial, if everyone taking the drug is also 20 years younger than the control group, you can't tell whether the recovery was caused by the drug or by age. Your "drug effect" is contaminated by an "age effect." A leaky prompt set contaminates the "refusal effect" with a "topic effect" in exactly the same way.

The defense is good experimental hygiene: the two piles should be matched in every respect *except* whether they trigger refusal — same topics, same styles, same lengths — so that refusal is the only thing left for the subtraction to find. (Section 17.3 returns to this as a named failure mode.)

> **Recap:** The refusal direction is found by averaging activations over a harmless pile and a refusal-triggering pile, then subtracting: `r_raw = mean_bad − mean_good`, normalized to `r_hat`. Averaging cancels everything the piles share,

> leaving the refusal signal. The catch — *direction leakage* — is that any other accidental difference between the piles also survives, so the piles must be well-matched.

---

7. Removing a direction from activations

We now have the refusal direction `r_hat` (Section 6) and the one-line formula to strip it out of any activation (Section 5.3). This section does two things: it rewrites that formula in **matrix form** (which looks scarier but is the same idea), and it introduces the crucial choice between removing refusal **live, as the model runs** versus **permanently, in the weights**. That choice is the bridge to a real, servable model.

7.1 Removing refusal live, one activation at a time

Recall the core operation. Given an activation `a`, subtract its refusal component:

```
a_ablated = a − (r_hat · a) r_hat
```

There is an equivalent way to write this using a single matrix, and it is worth meeting because the *weight* edit later uses the exact same object:

```
a_ablated = (I − r_hat r_hat^T) a
```

Let's decode that piece by piece — it is far friendlier than it looks:

- **`I` is the identity matrix.** Multiplying by `I` changes nothing; it is the matrix version of multiplying a number by 1. Think "leave the vector alone."
- **`r_hat r_hat^T` is an outer product.** It is a matrix built from the direction that does exactly one job: *take any vector, extract its refusal component, and hand that back*. (The `^T` just means `r_hat` written sideways, the bookkeeping that turns two vectors into a matrix.)
- **So `I − r_hat r_hat^T` reads as:** "keep the whole vector (`I`), then subtract its refusal part (`r_hat r_hat^T`)." Which is precisely `a − (r_hat · a) r_hat`, only packaged as one matrix.

That packaged matrix has a name: a **projection matrix**. Apply it and it flattens the refusal direction to zero while leaving every perpendicular (orthogonal) direction completely untouched.

> **Analogy — the photo filter that removes one color.** Picture a filter that strips all the red out of a photo while leaving every other color exactly as it was. `I − r_hat r_hat^T` is that filter, where "red" is the refusal direction. You can apply it to any image (activation) and get back the same image minus its red component. Crucially, the filter is fixed — it always removes the *same* color — which is why we can reuse it everywhere.

To use it live, you attach **runtime hooks**: small bits of code that intercept the residual stream at each layer while the model runs and apply the filter on the fly. The model never knows; from the outside it just stops refusing.

7.2 Why “live removal” is great for research but wrong for shipping

Removing refusal live at runtime has real advantages:

- **Reversible instantly:** detach the hooks and the original, refusing model is back. Nothing was permanently changed.
- **Perfect for experiments:** you can run the same prompt with and without the filter and compare, which is ideal for *studying* refusal.
- **The base model stays pristine:** useful if you want to keep one model and toggle behaviors.

But for actually *shipping* a model, the same approach is a burden:

- **It needs custom inference code.** The filter has to be re-applied at every layer, every token, every time. The model file alone is not enough — you must also ship and run the hook machinery.
- **It adds runtime overhead.** Extra work on every forward pass.
- **It does not produce a standalone model.** You cannot hand someone a single file and say “here, run it in any standard tool.”

> **Analogy — a live censor vs. editing the script.** Runtime hooks are like hiring a censor to sit beside the actor and bleep one word every time it is about to be said. It works, and you can fire the censor at any moment — but the censor must be present at *every* performance, forever. This project wanted to *ship the play*, not ship a censor with it. That is the motivation for the next subsection.

7.3 The key move: bake the removal into the weights (orthogonalization)

Here is the conceptual jump that turns abliteration from a research demo into a real model. Instead of *catching and removing* the refusal component every time it is produced, change the thing that *produces* it so it can never write that direction in the first place.

Recall from Section 3 that activations enter the residual stream through write-head matrices (`o_proj`, `down_proj`). If a matrix `W` writes to the residual stream, we pre-apply our filter to `W` itself:

```
W' = (I − r_hat r_hat^T) W
```

which, multiplied out, becomes the form you will actually see in code:

```
W' = W − r_hat (r_hat^T W)
```

What this formula does, piece by piece:

- `r_hat^T W` — this is a dot product between the refusal direction `r_hat` and *each column* of matrix `W`. The result is a row of numbers saying, for each possible input feature, “how much of that feature’s output would point along the refusal direction?”

- `r_hat (r_hat^T W)` — we multiply that result back by `r_hat`, spreading the “refusal amount” across all 12,288 output dimensions according to the refusal direction’s pattern. This builds a matrix of the same shape as `W` containing *only* the refusal-causing parts of `W`’s behavior.

- `W - [that]` — we subtract those refusal-causing parts, leaving a modified matrix `W'` that produces the same outputs as before *minus* anything in the refusal direction.

Now the *output* of `W'` is guaranteed to have no refusal component — not because we cleaned it up afterward, but because the writer is now physically incapable of producing it. The filter is applied **once, to the weights**, instead of **forever, to the activations**.

> **Analogy — retrain the pen, don't hire a proofreader.** Runtime hooks proofread every sentence and erase one word after it is written (Section 7.2). Weight orthogonalization instead reaches into the pen and removes its ability to write that word at all. Same end result on the page, but now the document is self-contained: no proofreader needs to travel with it. This is why the edit lives in the two write-heads and nowhere else — they are the pens.

> **Reminder — orthogonal:** two directions are *orthogonal* when they are perpendicular — their dot product is zero, so they don't overlap at all. "Weight orthogonalization" means making the write-head's output orthogonal to (perpendicular to) the refusal direction, i.e. having zero refusal component.

This is the whole reason the rest of the pipeline exists. A weight-baked edit is just a *different set of numbers in the same model file*, so it can be saved, merged, quantized, and served by ordinary tools — which is exactly what Sections 8–14 do.

> **Recap:** The removal formula can be written as one fixed **projection matrix** `I - r_hat r_hat^T`. Applied **live** to activations (runtime hooks) it is reversible and great for research but needs a "censor" present forever. Applied **once to the write-head weights** (`W' = W - r_hat(r_hat^T W)`, *orthogonalization*) it produces a self-contained, shippable model. This project chose the weight edit.

---

8. Baking the removal into weights

Section 7 gave us the weight edit `W' = W - r_hat (r_hat^T W)`. This section slows down to answer three honest questions a careful reader should be asking: *what does that edit do to the matrix's shape and meaning? why doesn't it wreck the model? and why isn't "remove the direction" the end of the story?* The answer to the third question is what makes the whole second half of the guide (Heretic) necessary.

8.1 What a write-head matrix actually is

A matrix is just a machine that turns one vector into another. A write-head `W` takes the block's internal result (its "input features") and turns it into a residual-stream vector (its "output"). We describe its size as:

```
W: d_out x d_in
   |       |
   |       +-- d_in : how many numbers come IN  (the block's internal
width)
   +---------- d_out: how many numbers go OUT  (the residual-stream
width)
```

For both `o_proj` and `down_proj`, the *output* lands in the residual stream, so `d_out` is the residual-stream width — `12,288` in this model. The refusal direction `r_hat` lives in that same output space (it is a direction *in the residual stream*), so it also has 12,288 components. This matching is what lets the edit make sense: `r_hat` and the matrix's output speak the same language.

> **Analogy — a translation desk.** Think of `W` as a translator who takes notes in one language (`d_in`) and writes them out in another (`d_out`, the shared document's language). The refusal direction is one forbidden *phrase* in the output language. The edit retrains the translator so that, whatever the input notes say, the output never contains that phrase — but every other phrase is translated exactly as before.

So the edit `W' = W − r_hat (r_hat^T W)` reads as: "keep the translator's behavior, but subtract off the part of every output that would have pointed in the refusal direction."

8.2 Why this preserves almost all of the model

This is the reassuring part. The edit is **not** a random shove to all of `W`'s numbers. It removes **exactly one direction** from the output of a **specific, small set** of matrices. Every direction orthogonal (perpendicular) to `r_hat` passes through completely unchanged — the projection only flattens the one refusal direction and leaves the other ~12,287 directions alone.

> **Analogy — confiscating one item at security, not emptying the bag.** A good airport scanner removes the single prohibited item and lets everything else through untouched. A bad one would dump the whole bag. Orthogonalization is the good scanner: one direction confiscated, the rest of the model's knowledge waved through. Compare this to fine-tuning, which reaches into *every* bag and rearranges the contents.

That said, honesty requires noting why it is not *perfectly* clean in practice — these caveats are exactly what later evaluation has to catch:

- **The measured direction is noisy.** It came from 400 prompts, not infinite data, so `r_hat` is an estimate.
- **Refusal may not be perfectly one-dimensional.** The Arditi result says *mostly* one direction, not *exactly*. Residual refusals can remain (ours: 6/100).
- **Some useful behavior may partly share the direction.** If "refusal" slightly overlaps with, say, "caution," removing it nicks caution too (this is the leakage risk from Section 6.5).
- **Quantization adds rounding noise** later in the pipeline (Section 14).

Even with all that, it remains a *vastly* more targeted operation than retraining — which is the whole appeal.

8.3 Why "just remove the direction" is not the end — strength

The idealized math says: remove the direction, done. Real models are messier, and this is the single most important practical refinement in the guide.

The issue is **how hard to remove it**, and *where*. Refusal is not represented with equal intensity in every one of the 88 layers — and as Section 8.2 admitted, the refusal direction slightly overlaps with useful behavior. So a one-size-fits-all removal is a blunt

instrument. We generalize the edit with a tunable **strength** knob:

```
W' = W − strength_layer ∗ r_hat (r_hat^T W)
              ^^^^^^^^^^^^^
              a dial we get to choose, separately for each layer
```

Read the dial like this:

- `strength = 0` -> the term vanishes, `W' = W`: **no change at all** (refusal untouched).
- `strength = 1` -> the exact projection from Section 7: **fully remove** the refusal component.
- `strength > 1` -> **over-remove**: push *past* zero, actively steering against refusal — powerful but destabilizing.

And the trade-off is unavoidable:

- **Strength too low** -> refusals remain. You under-did it.
- **Strength too high** -> you also tear out the useful behavior that shared the direction; the model gets incoherent or repetitive. You over-did it.

> **Analogy — antibiotic dose.** Too low a dose and the infection (refusal) survives. Too high and you harm the patient (the model's general ability). And different organs need different doses — here, different *layers* need different strengths. There is a therapeutic window, and it differs layer by layer.

Finding the right `strength_layer` for all 88 layers, separately for the attention and MLP write-heads, is not something you can eyeball. It is a search problem over many dials. **That search is exactly what Heretic automates** — and it is why the next sections move from "the math" to "the tool." Hold onto this strength dial; it is the thing Heretic spends 200 trials tuning.

> **Recap:** The edit removes one direction from a write-head's *output*, leaving the other ~12,287 directions intact — surgical, unlike fine-tuning. But "remove it" hides a dial: a per-layer **strength**. Too low leaves refusals; too high damages the model; the right value differs by layer. Tuning that dial across 88 layers is the job of Heretic.

---

9. Why `o_proj` and `down_proj` are the important matrices

We named these two matrices back in Section 3 and edited them in Sections 7–8. This section earns the choice properly: it explains why these two — out of the *seven* big matrices in every transformer block — are the right and complete place to cut, and why the others would be the wrong place.

> **Reminder — `o_proj` and `down_proj`:** the two matrices per layer that *write* into the residual stream — `o_proj` from the attention sub-block, `down_proj` from the MLP sub-block. They are the "pens" of Section 3. Everything else in a block only does internal scratch-work.

9.1 Each block has two sub-blocks, and each ends in a write

Every transformer block contains two engines:

- the **attention sub-block**, which gathers context *across* tokens ("what did the rest of the sentence say?"), and
- the **MLP (feed-forward) sub-block**, which processes each token's own state ("given all that, what is this?").

Each engine does a lot of internal computation, but each one ends by funneling its result through a *single final matrix* that deposits the answer into the shared residual stream:

- attention's final write -> **`self_attn.o_proj`**
- the MLP's final write -> **`mlp.down_proj`**

If refusal is a signal *written into* the residual stream (Section 4), then these two write points are the only doors it can come through. Guard the two doors and you have guarded the whole house.

9.2 Why not the other five matrices in the block?

A block actually contains around seven sizeable matrices. The other five — `q_proj`, `k_proj`, `v_proj` in attention, and `gate_proj`, `up_proj` in the MLP — are genuinely important, but they all do *upstream* work: they compute intermediate quantities that never touch the residual stream directly.

```
ATTENTION:  q_proj, k_proj, v_proj  --> [internal attention math] -->
o_proj --> residual stream
MLP:        gate_proj, up_proj      --> [internal MLP math]       -->
down_proj --> residual stream

^^^^^^^^                                                          only these
two actually WRITE
```

> **Analogy — a factory with one shipping door.** `q_proj`, `k_proj`, `v_proj`, `gate_proj`, and `up_proj` are workers on the assembly line, cutting and shaping parts inside the factory. `o_proj` and `down_proj` are the *shipping doors* — the only points where finished goods leave the building and enter the warehouse (the residual stream). To stop one product from reaching the warehouse, you do not need to retrain every worker; you put a check at the two shipping doors. Editing upstream workers would be indirect, hard to control, and would ripple through the internal math in unpredictable ways. Editing the doors is direct and contained.

This is the precise reason abliteration targets the write-back points and nothing else: it wants to prevent one direction from *entering* the residual stream, and the write-backs are where entering happens.

9.3 The project's actual numbers

This is not just theory — it is exactly what the run did. The adapter's metadata records the targets and size:

```
"r": 3,
"lora_alpha": 3,
"target_modules": [
  "self_attn.o_proj",
  "mlp.down_proj"
]
```

and Heretic's log confirms the count across the 88-layer model:

```
Transformer model with 88 layers
Abliterable components:
  attn.o_proj:  88 modules total   (one shipping door per layer,
attention side)
  mlp.down_proj: 88 modules total  (one shipping door per layer, MLP
side)
```

So the edit touched **176 matrices** — every shipping door in the building, and not one worker. (The `r: 3` and `lora_alpha: 3` lines are about *how the edit is stored*; that is the subject of Section 10.)

> **Recap:** Of the ~7 matrices per block, only `o_proj` (attention) and `down_proj` (MLP) write to the residual stream; the other five do internal scratch-work. Abliteration guards exactly those two "shipping doors," giving 176 edited matrices across 88 layers. Editing the upstream matrices would be indirect and uncontrolled.

---

10. Why abliteration naturally becomes a LoRA adapter

Here is a small miracle worth pausing on. The abliteration of a 128-billion-parameter model was saved as a file of just **69 MB**. How can an edit to such a giant model be so tiny? The answer is the word **low-rank**, and it is not a lucky compression trick — it falls directly out of the math we already did. This section explains *rank*, then shows that our edit was low-rank by construction, which is exactly why it fits the standard "LoRA" container.

10.1 What "rank" means (the idea behind the jargon)

The **rank** of a weight update is, intuitively, *how many independent directions of change it contains* — how much genuinely distinct "stuff" it does. A high-rank update reshuffles a matrix in many independent ways at once; a rank-1 update does just one thing, scaled up and down.

> **Analogy — a recipe vs. a cookbook.** A rank-1 update is a single recipe: "add salt." You can add a pinch or a spoonful (scale it), but it is still the one action. A high-rank update is a whole cookbook of unrelated recipes applied at once. Fine-tuning typically writes a cookbook (it changes the model in countless independent ways). Abliteration, as we will see, writes essentially one recipe per matrix — "remove the refusal phrase." That is why it is so compact.

10.2 LoRA: a container for low-rank updates

LoRA stands for **Low-Rank Adaptation.** Normally, changing a matrix `W` means storing a full-sized update `ΔW` (the Greek delta, meaning "the change in") that is just as big as `W` — wasteful if the change is simple. LoRA instead stores the update as the product of two *skinny* matrices `A` and `B`:

```
ΔW = (alpha / r) * B * A          and then        W_new = W + ΔW
```

The number `r` is the **rank** — literally how skinny `A` and `B` are, i.e. how many independent recipes you are allowing. `alpha` is just a scaling factor. Because `A` and `B` are tiny compared to `W`, the stored file is tiny too.

> **Analogy — storing the instructions, not the redrawn painting.** Imagine editing a huge mural. The full update `ΔW` is a second copy of the entire wall showing every changed pixel — enormous. LoRA instead stores a short instruction card: "shift this one band of colour by this much." If the edit really is that simple, the card is all you need, and it is thousands of times smaller. LoRA is the instruction card; `r` is how many lines the card is allowed to have.

LoRA is famous as a *fine-tuning* method, but nothing forces the update to come from training. It is simply a compact way to store *any* low-rank weight update — including one computed by geometry.

10.3 Abliteration's edit is low-rank by construction

Now the payoff. Take the weight edit from Section 8 and ask what the *change* `ΔW` is:

```
W' = W − r_hat (r_hat^T W)
ΔW = W' − W = − r_hat (r_hat^T W)
```

Look at the structure: `r_hat` is a column vector, and `(r_hat^T W)` works out to a row vector. A column times a row is an **outer product**, and an outer product is the very definition of a **rank-1** matrix — exactly one recipe. So *removing one direction is a rank-1 update*. Remove three directions and you get the sum of three rank-1 updates: rank 3.

> **This is the deep reason the abliteration fits in 69 MB.** The edit is not "a slightly different giant matrix." It is "subtract up to three specific directions" — and three directions take almost no space to write down. The compactness is not a separate optimization; it is the same one-direction-removal idea from Section 5, viewed as storage.

10.4 Why this project's adapter is rank 3 (and why that makes merging trivial)

The adapter config records exactly this:

```
r = 3            <- store up to 3 removed directions
lora_alpha = 3   <- scaling numerator
```

The LoRA formula scales the update by `alpha / r`. Here that is:

```
alpha / r = 3 / 3 = 1.0
```

A scale of exactly **1.0** is a small but real convenience: when the time comes to permanently fold the adapter into the base weights, there is no awkward scaling factor to track — you literally just add `B * A`. The merge script `scripts/merge_lora_bf16.py` does precisely that, written generally but evaluating to 1.0 here:

```
scaling = lora_alpha / r            # = 3 / 3 = 1.0
W = W + scaling * (B @ A)           # so effectively: W = W + (B @ A)
```

> **Reminder — what `B @ A` is:** the two skinny LoRA matrices multiplied back together to reconstruct the full-size update `ΔW`. "Merging" means computing that product once and adding it into `W`, after which the adapter is no longer needed as a separate file.

10.5 Two ways to use the result: adapter vs. merged

The low-rank edit can be delivered in two forms, and this project produced both:

- **Adapter form** — ship the tiny 69 MB LoRA *alongside* the untouched base model, and have the runtime add it on the fly. Portable, reversible, easy to toggle. This is the primary, archival deliverable.
- **Merged form** — permanently add `B * A` into the base weights to get one standalone model. Necessary because the eventual serving tool (llama.cpp) wants a single self-contained model file, not a base-plus-adapter pair.

The two key artifacts at the ends of this fork are:

```
adapters/mistral-medium-abliterated/                <- the portable
69 MB adapter
models/mistral-medium-abliterated-Q4_K_M.gguf       <- the merged,
quantized, servable model
```

Sections 13–14 follow the merged path all the way to a running server.

> **Recap:** *Rank* = how many independent changes an edit makes. Removing one direction is a rank-1 outer product, so removing three is rank 3 — naturally low-rank, which is why a 128B-model edit fits in 69 MB. LoRA is just the standard container for such an update (`ΔW = (alpha/r)·B·A`). With `alpha = r = 3` the scale is 1.0, so merging is a plain `W += B @ A`. You can keep the edit as a detachable adapter or merge it into a standalone model.

---

11. What Heretic adds beyond the basic method

We have, by now, the complete *recipe*: find the direction (Sections 5–6), project it out of the two write-heads (Sections 7–9), store it as a low-rank LoRA (Section 10). So what is left for a tool to do? Everything in the gap between "the math works in principle" and "the 128B model actually behaves well." That gap is the `strength` dial from Section 8.3 — and **Heretic** is the tool that turns the dial for you.

11.1 Why the basic method, by itself, is not enough

Pure directional ablation answers only the first question:

```
"Find a direction and remove it."  <- the math from Sections 5-9
```

It is silent on all the questions that actually decide whether you end up with a great model or a broken one:

```
- WHICH of the 88 layers should be edited?
- HOW STRONGLY should each one be edited?    (the strength dial, per layer)
- Should the attention pen (o_proj) and the MLP pen (down_proj) get DIFFERENT strengths?
- HOW do we trade "fewer refusals" against "more damage to the model"?
```

For a small model you might tune these by hand. For a *dense 128B* model, where a single test run is slow and the dials interact, hand-tuning is hopeless. There are effectively a handful of continuous dials per write-head type, producing a vast space of possible settings.

> **Analogy — a recording studio with a wall of dials.** The math from Sections 5–9 hands you a mixing board: one strength dial for the attention pen and one for the MLP pen *at every layer*, plus controls for how the strength rises and falls across depth. Turn them all to maximum and you get silence-of-refusals but a distorted, broken track. Turn them too low and the refusals are still audible. Somewhere in that wall of dials is a setting that sounds great. **Heretic is the audio engineer who systematically tries settings until the mix is right** — except it judges "right" with hard numbers, not ears.

11.2 Heretic's role, end to end

Heretic (by p-e-w) automates that search. In this project it performed, in order:

1. **Loaded the model** in a memory-feasible 4-bit format (we will see why in Section 13).
2. **Loaded the two prompt piles** — harmless and refusal-triggering (Section 6.1).
3. **Counted the model's initial refusals** — the `89/100` baseline.
4. **Computed per-layer refusal directions** — one `r_hat` per layer, via difference-of-means.
5. **Ran an optimization loop** over the strength dials, testing many settings.
6. **Reported a Pareto front** — the menu of best trade-offs between refusals and damage (Section 12).
7. **Restored the chosen trial's settings.**
8. **Saved the result as a LoRA adapter** (Section 10).

One distinction is so important it is worth stating loudly, because it is the most common misunderstanding about Heretic:

> Heretic uses an **optimizer**, but it is **not fine-tuning** the model. It never computes a gradient and never trains a weight. For each candidate setting of the dials, it *recomputes the same geometric edit* from Sections 7–9 and *measures the outcome*. It is searching over a few **edit hyperparameters**, not learning new weights.

> **Reminder — optimization vs. training.** *Training* (fine-tuning) nudges billions of weights using gradients to absorb new data. *Optimization* here means "try dial settings, keep the good ones" over a handful of numbers. Same word family,

> completely different scale and risk. The model's knowledge is never touched; only its refusal gate is dialed.

11.3 The actual run, in the log

Heretic's own log narrates the process:

```
Initial refusals: 89/100              <- baseline: the original model
refuses 89 of 100
Calculating per-layer refusal directions...   <- step 4 above
Resuming existing study.              <- the search (it can pause and
resume)
Optimization finished!                <- the search is done
```

And the setting it ultimately selected:

```
Trial 154
Refusals: 6/100
KL divergence: 0.1190
```

In plain terms: the chosen edit took refusals on the held-out set from **89/100 down to 6/100**, while keeping **KL divergence at 0.119** — a measure of collateral damage that we are about to define, and which 0.119 says is small. Section 12 unpacks what KL divergence is, why Trial 154 was chosen over alternatives, and the trade-off curve it sits on.

> **Recap:** The math says *what* edit to make; Heretic decides *how hard* to make it at each layer — the per-layer strength dials from Section 8.3. It loads the model, finds per-layer directions, then searches dial settings, scoring each by refusals and damage, and saves the winner as a LoRA. Crucially it *searches edit settings*, it does not *train weights*. This run went from 89/100 to 6/100 refusals at KL 0.119 (Trial 154).

---

12. Understanding MPOA, Optuna, KL divergence, and the Pareto front

This section is the conceptual climax of the guide. It introduces the four ideas behind *how Heretic chooses the edit*: **MPOA** (the method), **Optuna** (the search engine), **KL divergence** (how it measures damage), and the **Pareto front** (the trade-off it navigates). Each gets an analogy first. Take it in pieces.

12.1 MPOA: directional ablation with dials

This project's notes call Heretic's method **MPOA** — think of it as "the basic directional ablation from Sections 5–9, but *parameterized and optimized*." Where the basic method applies one fixed edit, MPOA exposes the per-layer strength dials and then searches them:

```
MPOA = measure per-layer refusal directions      (Section 6, once per
layer)
```

```
      + choose a per-layer STRENGTH profile      (the dials from
Section 8.3)
      + apply the geometric edit and TEST it     (count refusals +
measure damage)
      + search for a better strength profile      (repeat, getting
smarter each time)
```

The single new idea is that the strength is allowed to **vary by layer and by write-head type** (attention vs. MLP) instead of being one global number.

12.2 The per-layer strength profile — reading the dials

Why vary strength by layer? Because refusal is not represented equally at every depth. Recall from Section 3 the "document passed down a line of editors": early layers are still figuring out *what is being asked*, the middle layers carry the strongest *"this is a refusal context"* signal, and late layers are busy *committing to actual words*. So the refusal signal is loudest in the middle, and that is where ablation should push hardest.

Rather than store 88 separate numbers per write-head, MPOA describes the strength as a smooth **curve over depth**, defined by just four dials per write-head. Here is the selected trial's profile:

```
attn.o_proj.max_weight          = 1.16    <- peak strength (attention
pen)
attn.o_proj.max_weight_position = 60.25   <- the layer depth where the
peak sits
attn.o_proj.min_weight          = 0.93    <- the gentlest strength,
far from the peak
attn.o_proj.min_weight_distance = 40.73   <- how far from the peak
before it eases off

mlp.down_proj.max_weight          = 1.44  <- peak strength (MLP pen) -
pushed harder
mlp.down_proj.max_weight_position = 69.09 <- its peak sits a bit
deeper
mlp.down_proj.min_weight          = 1.28
mlp.down_proj.min_weight_distance = 50.45
```

Do **not** memorize these. Read them as a shape:

- **`max_weight`** is how hard the dial is turned at its strongest layer (recall from Section 8.3 that 1.0 = full removal, and >1.0 = over-removal, pushing past zero).
- **`max_weight_position`** is *which layer* that peak sits at (around layer 60 for attention, 69 for the MLP — i.e. in the deeper-middle, just as the editor analogy predicts).
- **`min_weight`** and **`min_weight_distance`** describe how the strength *tapers off* as you move away from the peak layer.

> **Analogy — applying sunscreen where the sun is strongest.** You do not coat your whole body in the same thick layer; you apply most where exposure is highest (shoulders, nose) and taper elsewhere. MPOA does the same: it finds the layers where the refusal "sun" is strongest (around 60–69 here) and concentrates the ablation there, easing off toward the shallow and deep ends. And the attention pen and MLP pen get *different* profiles, because they get different amounts of sun.

The four takeaways, which *are* worth remembering:

- **Different write-heads can get different strengths** (MLP peaks higher, at 1.44, than attention's 1.16).
- **Different layers can get different strengths** (the curve, not a constant).
- **The strongest ablation is centered at particular depths** (the `max_weight_position`).
- **The strength tapers away from those depths** (controlled by the two `min_*` dials).

12.3 Optuna: the engine that searches the dials

Heretic does not try every possible dial setting — there are far too many. It uses **Optuna**, an optimization library, with a sampler called **TPE (Tree-structured Parzen Estimator)**. You do not need the statistics; you need the behavior. TPE is a **smart search that learns from its own past attempts**:

```
1. Try some dial settings.
2. Measure how well each did (refusals + damage).
3. Learn which regions of dial-space looked promising.
4. Concentrate the next attempts in those promising regions.
5. Repeat, getting steadily smarter.
```

> **Analogy — "warmer, colder" with memory.** A brute-force search checks every spot in the room at random. TPE plays the children's "warmer / colder" game: each guess teaches it where to look next, so it homes in on the hidden prize far faster than random poking. It is the difference between guessing a password by brute force and getting a "you're close" hint after every try.

This run used **200 trials**, deliberately split:

```
first 60 trials:   random exploration   (cast a wide net, learn the
landscape)
remaining 140:     TPE-guided search    (exploit what was learned,
refine)
```

The early random phase matters: TPE needs a spread of examples before its "warmer/colder" instinct is any good. Cast the net wide first, then close in.

> **Reminder — what one "trial" is:** a single candidate setting of all the strength dials. Heretic builds the geometric edit for that setting, runs the held-out prompts through the edited model, and records two scores. No training happens inside a trial — it is build, test, score.

12.4 The two scores every trial gets (and why they fight)

Each trial is judged on exactly two numbers:

- **Refusal count** — of the 100 held-out refusal-triggering prompts, how many did the edited model *still* refuse? **Lower is better** (more abliterated).
- **KL divergence** — how much did the edited model's behavior on *benign* prompts drift from the original's? **Lower is better** (less collateral damage).

Both want to be low, but **they pull against each other**: turn the dials up and refusals fall *but* damage rises; turn them down and damage falls *but* refusals return. You cannot freely minimize both. That tension is the whole reason a *menu* of trade-offs (the Pareto front) exists rather than a single best answer — which is exactly what the rest of this section is about. First, the damage number itself.

12.5 KL divergence — the "how much did we damage it?" number

The refusal count is easy to picture. KL divergence is the subtle one, so let's build it from scratch — no probability theory assumed.

Recall from Section 2.1 that the model does not output a single next token; it outputs a **probability distribution** over all possible next tokens. On some benign prompt, the original and abliterated models each produce such a distribution:

```
Original model says the next token is:        Abliterated model says:
  "The"  0.30                                    "The"  0.29
  "A"    0.20                                    "A"    0.21
  "It"   0.10                                    "It"   0.11
  ...                                            ...
```

These two are nearly identical, which means our edit barely changed the model's ordinary behavior — good. **KL divergence is a single number that summarizes how different two such distributions are.** Identical distributions give KL = 0; the more they disagree, the larger KL grows.

> **Analogy — comparing two weather forecasters.** The original model is a trusted forecaster: "70% sun, 20% cloud, 10% rain." After surgery, does the new model still forecast like the trusted one? If it says "69% sun, 21% cloud, 10% rain," it has essentially the same judgement — KL near zero, no damage. If it suddenly says "10% sun, 10% cloud, 80% rain," its whole worldview has shifted — large KL, something is broken. KL divergence is the gap between the two forecasters' confidence, averaged over many forecasts. We deliberately measure it on **benign** prompts, because we want to know whether *normal* behavior survived — not whether refusals changed (that is what the refusal count is for).

So in our setting:

- **Low KL** -> on ordinary inputs the edited model still "thinks" like the original. Knowledge and reasoning likely intact.
- **High KL** -> the edit changed the model broadly. It is probably damaged: incoherent, repetitive, or worse at reasoning.

There is a rough but vital rule of thumb, noted right in Heretic's interface:

> **KL above about `0.5` generally signals serious capability damage.**

Keep that `0.5` threshold in mind, because it is the difference between a clever success and a cautionary tale. Trial 154, our pick, had KL **0.119** — comfortably in the healthy zone. By contrast, Trial 119 had KL **5.9257** — roughly *fifty times* the danger threshold. It achieved zero refusals, and it was rejected anyway, for reasons the next subsections make obvious.

> **Reminder — KL divergence:** one number measuring how far the edited model's next-token *probability distribution* has drifted from the original's, on benign prompts. 0 = identical; bigger = more collateral damage; >~0.5 = likely broken. It is the guardrail on the guardrail-removal.

12.6 The Pareto front — the menu of best trade-offs

Section 12.4 said the two scores fight each other. So there is no single "best" trial — there is a *menu* of trials, each striking a different balance. The **Pareto front** is the name for the *good* part of that menu: the set of trials where **you cannot improve one score without worsening the other.** Everything not on the front is simply dominated — some other trial beat it on both scores, so it is never worth choosing.

> **Analogy — shopping for a car on price vs. fuel economy.** Some cars are both pricier *and* thirstier than another option — nobody rational buys those; they are "off the front." The interesting choices are the ones where a cheaper car costs you fuel economy and an efficient car costs you money. That set of "can't-win-on-both" options is the Pareto front. Picking among them is a *judgement call about what you value*, not a math problem — and that is exactly the choice Heretic hands back to you.

Here is this project's actual front, from most aggressive to least:

```
Trial 119: Refusals  0/100, KL 5.9257   <- zero refusals, but model
DESTROYED
Trial 154: Refusals  6/100, KL 0.1190   <- *** the chosen one: strong
+ healthy ***
Trial 173: Refusals  7/100, KL 0.0657
Trial 160: Refusals  9/100, KL 0.0596
Trial 101: Refusals 13/100, KL 0.0434
Trial 193: Refusals 89/100, KL 0.0003   <- barely touched, basically
the original
```

Read down the list and feel the trade-off working: as refusals climb from 0 to 89, KL falls from 5.93 to nearly 0. Every row buys fewer refusals with more damage, or less damage with more refusals. That is a Pareto front in the flesh.

Trial 154 was chosen because it sits at the **elbow** of this curve — the point of diminishing returns:

```
- a huge refusal reduction (89 -> 6, about 93% gone)
- damage still low and safe (KL 0.119, well under the 0.5 line)
- NOT the absolute fewest refusals (that was Trial 119)
- NOT the absolute lowest KL (that was Trial 193)
- the best PRACTICAL balance of the two
```

> **Analogy — the knee in the curve.** Spending money to improve something usually has a "knee": up to a point you get big gains cheaply, then each extra bit of improvement costs wildly more. The elbow/knee of the Pareto curve is that sweet spot — squeezing out the last few refusals after Trial 154 would cost enormous amounts of KL (look at the jump to Trial 119), which is a terrible deal.

12.7 Why not just pick zero refusals?

It is tempting to grab Trial 119 — *zero* refusals sounds like total success. This is the most important judgement in all of abliteration, so be clear about why that is wrong.

The goal was never "eliminate refusals at any cost." It was:

```
Reduce refusal behavior WHILE PRESERVING the rest of the model.
```

Trial 119 hit 0 refusals but at KL **5.93** — about fifty times the damage threshold. That is not an uncensored model; it is a *broken* one that also happens not to refuse, the way a smashed radio also happens not to play adverts. Its refusals are gone because much of its competence is gone with them.

> A successful abliterated model is **not** the one with the fewest refusals. It is the one with the best **refusal-vs-capability trade-off** — strong refusal reduction bought with minimal damage.

> **Recap:** Each trial scores two fighting numbers: **refusal count** (lower = more abliterated) and **KL divergence** (lower = less damage; >~0.5 = likely broken). Because they trade off, Heretic returns a **Pareto front** — the menu of non-dominated options. The right pick is the **elbow** (Trial 154: 6/100 refusals, KL 0.119), not the extreme (Trial 119: 0 refusals but KL 5.93, a wrecked model). Fewest refusals is not the goal; best trade-off is.

---

13. The exact project pipeline in this repository

Everything so far has been the *idea*. This section is the *true story* of doing it to a real 128-billion-parameter model on one machine. Be warned of the central surprise: **almost none of the effort was the abliteration math.** The math is a few forward passes and some linear algebra. The work was getting a giant model to *fit*, and shuttling it between tools that each demand a different file format. Keep that in mind — it is the most practically useful lesson in the guide.

> **The whole journey at a glance.** Before the step-by-step, here is the relay race the model file runs. Each handoff exists because the *next* runner only accepts a particular baton (format). Section 14 explains each format; for now just watch the shape change:

```
  FP8 (original, ~126 GB)         "the model as shipped — too
awkward to edit directly"
    -> dequantize to BF16 (~238 GB)   "a clean, editable full-
precision copy"
      -> quantize to NF4 (~66 GB)        "small enough for
Heretic to load and abliterate"
        -> Heretic -> LoRA adapter (69 MB)  "the actual
abliteration, stored tiny"
          -> merge adapter into BF16          "a standalone
abliterated model"
            -> convert to GGUF f16               "the format
llama.cpp can read"
```

```
              -> quantize to Q4_K_M (~70 GB)        "small
enough to serve comfortably"
                -> serve with llama.cpp                "a
running, abliterated chatbot"
```

Every arrow is a forced move, not a preference. The rest of this section explains *why* each one was forced.

13.1 The one constraint behind every decision: unified memory

The work ran on an NVIDIA DGX Spark / GB10-class machine with roughly:

```
121 GB of UNIFIED memory shared by the CPU and GPU
```

The word **unified** is the crux. On a typical gaming PC, the GPU has its own separate VRAM (say 24 GB) *plus* the system has separate RAM (say 64 GB). Here there is **one shared 121 GB pool** for everything. There is no "offload it to the other memory" escape hatch — if the model plus its working space exceeds 121 GB, the job dies.

> **Analogy — one studio apartment, not a house.** A normal workstation is a house: the GPU has its own room (VRAM) and the CPU has another (RAM), and you can shuffle furniture between rooms. The Spark is a single studio apartment of 121 GB. Everything — model, scratch space, operating system — shares that one floor. The model in its native form is a piece of furniture too big for the door, which is why the entire pipeline is about *reshaping the furniture to fit*.

This single fact explains the dequantize/quantize gymnastics that follow.

13.2 Where it all lives

The project sits at:

```
/home/joedowling/Projects/heretic-mistral-medium
```

and the two artifacts that matter at the end are the ones from Section 10.5:

```
adapters/mistral-medium-abliterated/            <- the portable 69 MB
abliteration
models/mistral-medium-abliterated-Q4_K_M.gguf   <- the final servable
model (~70 GB)
```

13.3 Phase 1 — first, just get the original model to run at all

Before changing anything, the project needed proof it could *run* Mistral Medium 3.5 128B on this hardware. A pre-made **Q4_K_M GGUF** build (~70 GB) was downloaded, because the original full-size format would not load comfortably in 121 GB.

But running it exposed the first toolchain landmine: **Ollama crashed** on this model's brand-new `mistral3` architecture. The fix was to build **llama.cpp** from source, whose more mature kernels served the model fine.

> **Reminder — GGUF and llama.cpp:** *GGUF* is the model file format used by *llama.cpp*, a fast, memory-efficient inference engine. They are the *serving* side of the story; Heretic does not use them. We define both properly in Section 14.

> **Lesson — the newest tool is not always the working tool.** Ollama is newer and slicker; llama.cpp is older and battle-tested. For a cutting-edge architecture, maturity beat novelty. With large models, the *engine* matters as much as the model file.

13.4 Phase 2 — make an editable copy (FP8 -> BF16)

Now the catch that shapes everything. Heretic edits weights through the **HuggingFace/transformers** ecosystem — *not* GGUF. And to fit in memory, it wants to load the model in 4-bit (NF4) via a library called **bitsandbytes**. But the original weights shipped in **FP8**, and:

```
FP8 model + bitsandbytes 4-bit  =  REJECTED ("you can't double-
quantize")
```

> **Analogy — you can't re-compress a compressed file cleanly.** FP8 is already a compressed (quantized) form. Asking bitsandbytes to compress it *again* into 4-bit is like trying to vacuum-pack an already vacuum-packed bag — the tools refuse, because stacking lossy compression on lossy compression is a mess. The clean fix is to first *unpack* it back to full size, then re-pack it the way you actually need.

So the project first **dequantized FP8 back up to BF16** (full-ish precision), using a streaming script:

```
models/Mistral-Medium-3.5-128B            (original FP8)
  -> scripts/dequant_fp8_to_bf16.py
  -> models/Mistral-Medium-3.5-128B-bf16  (~238 GB)
```

Conceptually the dequantizer just undoes the compression, multiplying each stored FP8 weight by its saved scale factor:

```
W_bf16 = W_fp8 * scale
```

The resulting BF16 model is huge — about **238 GB** — far bigger than the original. That sounds backwards, but BF16 is the *lingua franca for editing and merging*: clean, standard, and accepted by every conversion tool. It is the "unpacked" form you do surgery on, even though it will not itself fit in memory all at once.

> **Reminder — FP8 vs. BF16:** *FP8* (8-bit) is the small, shipped form. *BF16* (16-bit) is the larger, full-precision form used for editing. Bigger = more faithful and tool-friendly, but heavier. Section 14 has the details.

13.5 Phase 3 — shrink it so Heretic can load it (BF16 -> NF4)

The 238 GB BF16 model obviously cannot sit inside 121 GB. So, just for the abliteration run, the project re-compressed it to **NF4**, a 4-bit format for the bitsandbytes loader:

```
models/Mistral-Medium-3.5-128B-bf16
  -> scripts/quantize_nf4_streaming.py
```

```
  -> models/Mistral-Medium-3.5-128B-nf4
```

In NF4, the model loads at about **66 GB** resident — finally small enough to abliterate inside the 121 GB budget.

> **Reminder — why NF4 is temporary:** NF4 is the *editing-time* format, used only so Heretic can fit the model in memory. It is not the final serving format (that will be Q4_K_M GGUF). Different jobs, different compressions.

This is the "re-pack it the way you actually need" step from the Phase 2 analogy: FP8 -> unpack to BF16 -> repack to NF4. Three formats, and we have not even started the abliteration yet.

13.6 Phase 4 — the actual abliteration (run Heretic)

This is the step everything else exists to enable, and — fittingly — it is the one we have already fully explained in Sections 11–12. Heretic loaded the 66 GB NF4 model, computed a refusal direction per layer (Section 6), and ran the 200-trial MPOA search (Section 12). The log summarizes the run:

```
88 transformer layers
88 attn.o_proj modules                          \  the 176 write-heads
from Section 9
88 mlp.down_proj modules                        /
400 harmless direction-finding prompts          \  the piles from Section
6.1
400 refusal-triggering direction-finding prompts /
100 harmless eval prompts                       \  the held-out grading
set
100 refusal-triggering eval prompts             /
Initial refusals: 89/100                        <- the baseline
Selected result: Trial 154, 6/100 refusals, KL 0.1190   <- the chosen
elbow
```

Then one more memory wall: Heretic's *built-in* "merge and save a full model" path needed more memory than the box had. So Heretic saved only the tiny **LoRA adapter** (69 MB) — which, happily, is the portable form we wanted anyway (Section 10.5). The merge would be done by hand later, in the memory-friendly BF16 form.

13.7 Phase 5 — turn the adapter into a standalone model (merge into BF16)

The 69 MB adapter is portable, but it is only *half* a model — it is the "subtract these directions" instruction card that must sit on top of a base model. The eventual server (llama.cpp) wants **one self-contained model file**, not a base-plus-adapter pair. So the adapter was permanently folded into the BF16 base:

```
models/Mistral-Medium-3.5-128B-bf16
  + adapters/mistral-medium-abliterated
  -> scripts/merge_lora_bf16.py
  -> models/Mistral-Medium-3.5-128B-abliterated-bf16
```

This is exactly the merge math from Section 10.4 — streamed shard by shard to respect memory:

```
W = W + scaling * (B @ A)        with   scaling = lora_alpha / r = 3 /
3 = 1.0
so effectively:  W = W + (B @ A)
```

> **Reminder — merging:** computing `B @ A` (the adapter's full-size update) once and adding it into the base weights, so the abliteration becomes a permanent part of a single model file. After this, the adapter is no longer needed to run the model.

The result is a standalone, abliterated, full-precision (BF16) model — correct, but ~238 GB and in the wrong format for serving. Two more steps fix that.

13.8 Phase 6 — convert to llama.cpp's format (BF16 -> GGUF f16)

The merged model is HuggingFace safetensors; llama.cpp reads **GGUF**. So it was converted with llama.cpp's own tool:

```
HF safetensors BF16 merged model
  -> convert_hf_to_gguf.py
  -> GGUF f16 model
```

This is a *format* change, not a *size* change — still full-ish precision, just repackaged into the container the serving engine understands.

> **Reminder — GGUF:** the on-disk model format that llama.cpp consumes. Think of this step as putting the same model into a different box so a different tool can open it.

13.9 Phase 7 — compress for serving (GGUF f16 -> Q4_K_M)

A full-precision GGUF is still far too large to serve comfortably in 121 GB. The final compression is to **Q4_K_M**, llama.cpp's high-quality 4-bit format. The quantization log shows the dramatic shrink:

```
model size  = 238472.30 MiB     (~238 GB, full precision)
quant size  =  71419.92 MiB     (~70 GB, Q4_K_M)
```

That is the final servable artifact, about **70 GB** — the same size as the Q4_K_M build we downloaded in Phase 1, but now it is the *abliterated* model rather than the original.

> **Reminder — two different 4-bit formats, two different jobs.** NF4 (Phase 3) was for *editing* with bitsandbytes; Q4_K_M (here) is for *serving* with llama.cpp. The model was effectively quantized twice, for two different tools. (Section 14.5 and Section 16.5 discuss the small quality cost of that double trip.)

13.10 Phase 8 — serve it

Finally, llama.cpp serves the finished model as a local API:

```
~/Projects/llama.cpp/build/bin/llama-server \
  -m ~/Projects/heretic-mistral-medium/models/mistral-medium-
        abliterated-Q4_K_M.gguf \
  -ngl 99 -c 8192 --host 0.0.0.0 --port 8080 --jinja
```

And the memory budget from Section 13.1 finally balances:

```
~76 GB used of 121 GB
  ~70 GB model weights
   ~6 GB KV-cache / compute working space
~45 GB free
```

The furniture fits through the door, with room to spare.

> **Recap:** The pipeline is a forced relay of formats driven by one 121 GB memory limit: download a Q4_K_M just to prove it runs (and discover Ollama fails, llama.cpp works); dequantize FP8 -> BF16 to get an editable copy; quantize BF16 -> NF4 so Heretic can load it; run Heretic to get a 69 MB LoRA; merge that into BF16; convert to GGUF; quantize to Q4_K_M; serve. The abliteration itself (Phase 4) was the *easy* part — the rest was logistics.

---

14. The model formats: FP8, BF16, NF4, GGUF, and Q4_K_M

Section 13 threw five format names at you. They are the single biggest source of confusion in this whole project, so this section slows down and gives each one a job description. First, the idea underneath all of them.

14.0 What "quantization" even means (and two axes to keep separate)

Every weight in the model is a number. **Quantization** means storing those numbers with *fewer bits* — fewer "digits of precision" — to make the file smaller, at the cost of a little accuracy.

> **Analogy — decimal places, or JPEG quality.** Storing `3.14159265` is precise but bulky; rounding to `3.14` is smaller but slightly off. Quantization is choosing how many "decimal places" to keep for billions of weights. It is the same trade-off as saving a photo at high vs. low JPEG quality, or music at high vs. low bitrate: smaller file, slightly degraded fidelity. A model has so many weights that dropping from 16 bits to 4 bits roughly *quarters* the file — turning hundreds of gigabytes into tens — while usually keeping the model's behavior almost intact.

Now the key to not getting lost: these five names actually live on **two different axes**, and people confuse them constantly.

- **Precision (how many bits per weight):** FP8 (8-bit), BF16 (16-bit), NF4 and Q4_K_M (4-bit). This is the "how much fidelity" axis.
- **Container (which file format / which tool reads it):** HuggingFace *safetensors* (used by transformers, bitsandbytes) vs. *GGUF* (used by llama.cpp). This is the "which ecosystem" axis.

> **Analogy — resolution vs. file type.** "4K vs. 1080p" (resolution) is a different question from ".mp4 vs. .mov" (container). A video can be any resolution in any container. Likewise BF16 is a *precision*; GGUF is a *container*; Q4_K_M is a *precision inside the GGUF container*. The pipeline changes both axes at different steps, which is exactly why it feels confusing — keep asking "is this step changing fidelity, or changing tool?"

With that, here is each format's job.

14.1 FP8 — the shipped form (8-bit; "as delivered")

The original Mistral Medium weights arrived in an **FP8** (8-bit floating point) format, with accompanying scale tensors. Its only role here was *source material*. The problem (Section 13.4): you cannot cleanly stack bitsandbytes' 4-bit on top of an already-FP8 model — tools reject the double-compression — which forced the detour up to BF16.

> **Reminder — FP8's role:** the starting point, and the reason the pipeline begins with an "unpack" step.

14.2 BF16 — the editing form (16-bit; "the workbench")

BF16 (bfloat16, 16-bit floating point) is the full-ish-precision reconstruction. It is the format you do *surgery* on:

- straightforward, standard tensor math (easy to apply the LoRA update),
- accepted by essentially every conversion tool,
- the common denominator everything else converts to or from.

Its only downside is size: ~238 GB for this model. It is the clean workbench you assemble on, not the thing you ship.

> **Reminder — BF16's role:** the editable/mergeable "workbench" format. Big but faithful and tool-friendly.

14.3 NF4 — the editing-time shrink (4-bit; "fits for Heretic")

NF4 is a 4-bit format used by the bitsandbytes loader. Its sole job was to shrink the model enough (~66 GB) that **Heretic could load it** within the 121 GB budget during abliteration. It is purely an *intermediate, editing-time* format — never shipped.

> **Reminder — NF4's role:** "make it fit so Heretic can run." Temporary, editing-side, bitsandbytes.

14.4 GGUF — the serving container ("the box llama.cpp opens")

GGUF is not a precision; it is the **file format (container)** that llama.cpp reads. The merged BF16 model was *converted* into GGUF so the serving engine could open it at all. A GGUF can hold weights at various precisions (full or quantized).

> **Reminder — GGUF's role:** the serving-side *container*, consumed by llama.cpp. A change of box, not (by itself) of fidelity.

14.5 Q4_K_M — the serving shrink (4-bit, inside GGUF; "ships to users")

Q4_K_M is llama.cpp's high-quality 4-bit quantization, applied *within* the GGUF container. It is the **final serving artifact**, shrinking the model from ~238 GB to ~70 GB so it serves comfortably with headroom.

> **Reminder — Q4_K_M's role:** the final, compressed, *servable* form. Serving-side counterpart to NF4's editing-side role.

> **Note — quantized twice, on purpose.** The model was 4-bit-quantized *twice* by two different tools for two different jobs: NF4 for editing, Q4_K_M for serving. Each adds a little rounding noise. It works fine here because, as Section 8.2 argued, removing a *direction* is a coarse, robust change that survives rounding. Section 16.5 says how this was verified.

14.6 The whole format journey on one page

```
Original FP8 HF safetensors          (8-bit; shipped form)
  -> dequantize to BF16              (16-bit; editable workbench, ~238
GB)
  -> quantize to NF4 for Heretic     (4-bit; fits in memory, ~66 GB)
  -> Heretic produces LoRA adapter   (the abliteration itself, 69 MB)
  -> merge LoRA into BF16 base        (standalone abliterated model)
  -> convert merged BF16 to GGUF f16  (change container -> llama.cpp)
  -> quantize GGUF to Q4_K_M          (4-bit; final servable form, ~70
GB)
  -> serve with llama.cpp             (running chatbot)
```

Read the journey as alternating moves on the two axes: change *fidelity* (FP8->BF16->NF4), then do the work, then change *container* (safetensors->GGUF), then change *fidelity* again (f16->Q4_K_M).

> **Recap:** Quantization trades bits (fidelity) for size. Keep two axes separate: *precision* (FP8/BF16/NF4/Q4_K_M) vs. *container* (safetensors vs. GGUF). FP8 = shipped, BF16 = editable workbench, NF4 = editing-time shrink for Heretic, GGUF = serving container, Q4_K_M = serving-time shrink. The model is deliberately 4-bit-quantized twice — once per tool — and the abliteration is robust enough to survive it.

---

15. What each important file in this repo does

Now that you understand the pipeline, the repository's files stop looking like a random pile and start looking like *stations along the journey from Section 13*. It helps to group them by role:

- **The two deliverables:** the adapter (15.2) and the final GGUF (15.3) — the things you actually keep.
- **The three format-moving scripts:** dequant, NF4 quantize, merge (15.4–15.6) — one per arrow in the pipeline.

- **The two logs:** the receipts proving what happened (15.7–15.8).
- **The docs:** this guide and its companions (15.1, 15.9).

15.1 `ABLITERATION_GUIDE.md`

The original concise technical guide. It is accurate and useful *once you already know the vocabulary*. This file, `ABLITERATION_EDU.md`, is the gentle on-ramp to that same destination — same facts, more scaffolding.

15.2 `adapters/mistral-medium-abliterated/`

The Heretic output adapter.

Important files:

```
adapter_config.json
adapter_model.safetensors
```

Important adapter facts:

```
PEFT type: LORA
rank r: 3
alpha: 3
target modules: self_attn.o_proj, mlp.down_proj
base model path: models/Mistral-Medium-3.5-128B-nf4
```

15.3 `models/mistral-medium-abliterated-Q4_K_M.gguf`

The final servable abliterated model.

This is the artifact you can serve with llama.cpp.

15.4 `scripts/dequant_fp8_to_bf16.py`

Streams through the original model shards and reconstructs BF16 weights from FP8 weights plus scale tensors.

Purpose:

```
make an editable/mergeable BF16 base model
```

15.5 `scripts/quantize_nf4_streaming.py`

Streams through the BF16 model and creates a bitsandbytes NF4 model.

Purpose:

```
make a memory-feasible model for Heretic
```

Important detail: the script determines which linear modules transformers would quantize and skips modules that should not be converted.

15.6 `scripts/merge_lora_bf16.py`

Streams through BF16 model shards, loads LoRA tensors, and applies:

```
W = W + scaling * (B @ A)
```

Purpose:

```
create a standalone abliterated BF16 model before GGUF conversion
```

15.7 `heretic.log`

The main record of the Heretic abliteration run.

It contains:

- model/layer summary,
- prompt loading,
- initial refusal count,
- optimization result,
- Pareto front,
- selected trial,
- save behavior.

15.8 `quantize.log`

Record of llama.cpp quantization to Q4_K_M.

Useful for confirming final size and successful quantization.

15.9 `README.md` and `CLAUDE.md`

These appear to include earlier project notes and some legacy references. For the Mistral Medium 3.5 run, `ABLITERATION_GUIDE.md`, logs, scripts, and adapter metadata are more authoritative.

---

16. Evaluation: how to tell whether the model survived

Recall the two goals that fight each other (Section 12.4): **reduce refusals** *and* **preserve everything else**. A claim of success needs evidence for *both* — and no single number proves both at once. So we use four complementary lenses. Each catches something the others miss.

> **Analogy — a medical checkup, not a single thermometer.** A doctor does not declare you healthy from temperature alone; they check temperature, blood pressure, blood work, *and* talk to you about how you feel. Evaluating an abliterated model is the same: refusal rate, KL divergence, benchmarks, and hands-on chat are four instruments, and a problem invisible to one often shows up in another. Trusting only one number is how you ship a broken model that "scored fine."

16.1 Lens 1 — Refusal rate (did we achieve the goal?)

The refusal rate is how often the model still refuses on the held-out evaluation set. It measures whether the abliteration *worked*:

```
before:                 89/100 refusals
after (Trial 154):       6/100 refusals
```

A drop from 89 to 6 is a large, real behavioral change. **But on its own it proves nothing about damage** — Trial 119 hit 0/100 and was a wreck. That is why we need Lens 2.

16.2 Lens 2 — KL divergence (did we damage normal behavior?)

> **Reminder — KL divergence:** one number for how far the edited model's next-token probabilities drifted from the original's *on benign prompts*; 0 = identical, >~0.5 = likely broken (Section 12.5).

This is the counterweight to refusal rate. Ours was:

```
Trial 154 KL: 0.1190     (well under the ~0.5 danger line)
```

Low KL is reassuring — but it is still only a *proxy*. It samples next-token behavior on a limited set of benign prompts; it cannot, by itself, confirm that long reasoning or coding still work. Hence Lenses 3 and 4.

16.3 Lens 3 — Benchmark retention (did specific skills survive?)

KL is a broad average; benchmarks are targeted skill tests. The thorough check is to run standard evaluations on the *original* and *abliterated* models and compare scores across:

- factual QA, reasoning, coding, math, summarization, instruction following, and long-context behavior.

If the scores barely move, that is strong evidence the edit preserved real capability — it confirms KL's promise with concrete tasks.

> **Analogy — a student after minor surgery.** KL says "their overall vital signs look normal." Benchmarks are "re-sit the maths, history, and coding exams and check the marks didn't drop." You want both: healthy vitals *and* unchanged grades.

16.4 Lens 4 — Qualitative probing (does it still *feel* right?)

Numbers can hide subtle damage, so you must actually *chat* with the model and watch for:

- incoherence or rambling, repetition or loops, degraded reasoning,
- odd persona shifts, weirdly excessive compliance, loss of nuance,
- broken formatting, or ignored instructions.

> **Analogy — a test drive after the mechanic signs off.** The diagnostics can read "all green" while the car still pulls to one side or rattles at speed. You only catch that by driving it. Some abliteration damage (a flattened writing style, a tendency

> to loop) is exactly this kind of "feel" problem that no single metric flags.

16.5 Don't forget: evaluate the *final* artifact, not just an intermediate

A subtle trap: it is tempting to evaluate the adapter or the BF16 merged model and call it done. But what users actually run is the **Q4_K_M GGUF**, which went through an extra round of 4-bit quantization (Section 14.5). Quantization *could*, in principle, blunt a marginal abliteration. So the served artifact must be tested directly. In this project it was, and the abliteration survived the quantization — but "it should survive" is a hypothesis, and you verify hypotheses.

16.6 The ideal evaluation stack

Putting the lenses together, a rigorous process checks the behavior at *every* stage where it could change:

```
1. Heretic's refusal/KL eval during the optimization itself (built
in).
2. Manual sanity tests on base + adapter.
3. Manual sanity tests on the merged BF16 model.
4. Manual sanity tests on the final Q4_K_M GGUF (what users get).
5. Standard benchmark comparison vs. the original model.
6. Longer qualitative chat sessions.
```

> **Recap:** Success means fewer refusals *and* preserved capability, and no single metric proves both. Use four lenses — refusal rate (goal achieved?), KL divergence (broad damage?), benchmarks (specific skills?), and hands-on chat (subtle "feel" issues?) — and run them on the *final served artifact*, not just an intermediate.

---

17. Failure modes and troubleshooting lessons from this run

Things go wrong in two distinct places, and it helps to keep them apart:

- **Method failures (17.1–17.3)** — the abliteration itself misses, in one of three ways. Notice they map exactly onto the strength dial from Section 8.3 and the leakage warning from Section 6.5: turn the dial too far, not far enough, or aim it at the wrong target.
- **Engineering failures (17.4–17.6)** — the math was fine, but the *logistics* (tools, memory, speed) bit back. As Section 13 warned, this is where most real time is lost.

17.1 Over-ablation (the dial turned too far)

You pushed the strength too high. Refusals vanish, but you tore out useful behavior that shared the direction.

Symptoms: very low refusal count *together with* high KL, incoherence, repetition, worse reasoning, degraded instruction-following.

```
Trial 119: 0/100 refusals, KL 5.9257     <- NOT a success; a damaged
model
```

> **Analogy — weeding with a flamethrower.** The weeds (refusals) are certainly gone, but so is the lawn. Zero refusals achieved by scorching the model is not a win — this is the cautionary tale from Section 12.7.

17.2 Under-ablation (the dial not turned far enough)

The opposite miss: too gentle. The model stays pristine but still refuses.

Symptoms: very low KL, behavior close to the original, but refusals remain high.

```
Trial 193: 89/100 refusals, KL 0.0003     <- model preserved, but goal
not met
```

> The two extremes of the Pareto front (Section 12.6) *are* these two failure modes. The whole art is landing at the elbow between them.

17.3 Direction leakage (the dial aimed at the wrong thing)

The most insidious failure: the refusal direction `r_hat` you measured was contaminated with some *other* concept, so removing it also removes that concept. This is the entanglement risk from Section 6.5, now listed as a thing that actually bites.

> **Reminder — direction leakage:** when the harmless and refusal-triggering prompt piles differ in some way *besides* refusal (topic, style, length), that difference rides along in `r_hat` and gets ablated too. You aimed at "refusal" but hit "refusal + chemistry knowledge."

Common causes: poor prompt contrast, too few prompts, topic/domain imbalance between the piles, dataset artifacts, or the two sets differing in writing style as well as refusal status.

Mitigations: use well-matched, balanced prompt sets; always grade on *held-out* prompts; watch KL; run real capability benchmarks (Section 16.3); and refuse to pick over-aggressive trials.

17.4 Toolchain incompatibility (the format/tool boundary)

This run hit a wall of tooling realities, none of them about the algorithm:

- Ollama crashed on this `mistral3` architecture (Section 13.3).
- transformers needed a newer version for the model's config; older versions did not support it.
- bitsandbytes could not be stacked directly on FP8 (Section 13.4) — the "no double-quantize" wall.
- Heretic's built-in merge path did not fit in memory (Section 13.6).
- Conversion and quantization required llama.cpp's specific tools.

> **Lesson — projects are won or lost at the boundaries.** The math worked the first time. The *days* went to format conversions and tool compatibility. With large models, budget most of your effort for the seams between tools, not the algorithm

> in the middle.

17.5 Memory spikes (steady-state is not peak)

A trap specific to the unified-memory constraint (Section 13.1): a model can fit *once loaded* yet still crash *during* loading or merging, because those operations transiently need roughly double the memory.

```
steady-state memory < available memory      <- true...
peak memory during loading/conversion < available memory   <- ...does NOT follow!
```

> **Analogy — fitting through the doorway, not just into the room.** A sofa may fit comfortably in the room (steady state) yet not fit through the doorway at the angle required to get it in (the loading spike). The project survived these doorways with *streaming* scripts (process one shard at a time) and *swap* (160 GB + 256 GB files) so transient spikes page to disk instead of killing the job.

17.6 Dense-model throughput (why it was slow)

Mistral Medium 3.5 128B is a **dense** model: every token uses essentially *all* 128B parameters. That is why the Heretic run crawled at ~3 tokens/second.

> **Analogy — dense vs. Mixture-of-Experts (MoE).** A dense model is a company where *every* employee attends *every* meeting — thorough but slow. An **MoE** model of similar total size keeps just as many employees on payroll (same memory footprint) but sends only a few specialists to each meeting (few *active* parameters per token) — far faster on hardware like this, which was built to *hold* a big model while *computing* only a slice of it.

> **Lesson:** *total* parameter count drives storage and memory pressure; *active* parameter count drives speed. If throughput matters more than this specific model, an MoE is the better-matched target for this box.

> **Recap:** Two failure families. Method failures track the strength dial and prompt quality: **over-ablation** (too strong, model damaged — Trial 119), **under-ablation** (too weak, refusals remain — Trial 193), and **direction leakage** (wrong target, from mismatched prompt piles). Engineering failures are logistics: **toolchain** mismatches, **memory spikes** during loading (steady-state ≠ peak), and slow throughput from a **dense** architecture.

---

18. Responsible operation

We have spent the whole guide learning to *remove* a safety layer. This section is the necessary counterweight: what that means for the risk you now own. Abliteration is **dual-use** — the very same technique serves legitimate research and removes a guardrail — so the responsibility cannot be skipped.

> **Analogy — removing a car's speed limiter.** A factory speed limiter is sometimes genuinely in the way (on a closed track, in an emergency-services vehicle), and removing it is a legitimate engineering act. But once it is gone, *every* decision

> about speed falls to the driver — the car will no longer stop you. Abliteration removes the model's internal "limiter." That can be the right call for the uses below, but from that moment, the judgement that the model used to apply is **entirely yours**.

Appropriate uses include:

- interpretability research (this is literally the Arditi et al. research program),
- safety and red-team research — studying how robust alignment really is,
- studying the robustness of alignment methods generally,
- reducing *over-refusal* for legitimate local use (many aligned models refuse benign requests — security education, fiction, medical information),
- controlled, private deployments where the operator knowingly accepts the risk.

Operational recommendations:

- **Restrict access:** do not casually expose the model to untrusted users.
- **Log and monitor usage:** especially if it is served on a network.
- **Keep it local/private by default:** the final model is uncensored *relative to the base*.
- **Add external policy controls if needed:** abliteration removed an *internal* safety layer, so any needed safety must now come from the *outside* — filters, system prompts, human review.
- **Do not treat outputs as safe or correct:** the model can still hallucinate, comply with bad ideas, or give low-quality advice. Removing refusal does not add truthfulness.

The single principle that ties it together:

> Removing a model's refusal behavior does not delete the risk — it **moves** the risk from the model's built-in alignment layer to the human, operator, and system around it. You are now the guardrail.

---

19. Glossary

Every term in one place, in alphabetical order, each with a pointer back to where it was built up. Use this as a reference when a word reappears and you want the one-line version.

Ablation

Removing part of a system to study or change its behavior. Here: removing a *direction* from activations or weights. (Section 5)

Abliteration

Ablate + *obliterate*. Ablation aimed at deleting a learned behavior — here, refusal — by finding its direction and editing it out of the weights. (Section 1)

Activation

A vector of numbers inside the model during a forward pass — the model's temporary internal state for one token. (Section 2.2)

Alignment

The extra training (SFT, RLHF, DPO) that turns a raw text-predictor into a helpful, harmless assistant — and that *installs* refusal in the first place. (Section 4.1)

BF16

Bfloat16, a 16-bit floating-point precision. The full-ish-precision "workbench" format used for editing and merging; large (~238 GB here) but tool-friendly. (Section 14.2)

bitsandbytes

A library for memory-efficient quantized model loading (4-bit/8-bit). The tool that loads the NF4 model for Heretic. (Sections 13.4, 14.3)

Dense (vs. Mixture-of-Experts)

A *dense* model uses all its parameters for every token (thorough but slow). An *MoE* uses only a few "expert" parameters per token (same memory, much faster). Mistral Medium 3.5 is dense, hence ~3 tok/s. (Section 17.6)

Difference of means

The method for finding the refusal direction: average activations on the refusal-triggering pile minus average on the harmless pile, then normalize. (Section 6.3)

Direction

An orientation (an arrow's "which way," ignoring length) in activation space. The key idea of the guide: behaviors live in directions, not single numbers. (Sections 2.2, 5)

Dot product

A single number measuring how much two vectors point the same way; `r_hat · a` = "how much refusal is in activation `a`." (Section 5.2)

`down_proj` (mlp.down_proj)

The MLP "down projection" — one of the two write-heads per layer; writes the MLP's result into the residual stream. An abliteration target. (Sections 3.3, 9)

FP8

8-bit floating-point precision. The format the original weights shipped in; cannot be cleanly re-quantized by bitsandbytes, forcing the BF16 detour. (Sections 13.4, 14.1)

GGUF

The on-disk model *container* format read by llama.cpp. A change of "box," separate from precision. (Section 14.4)

Heretic

The automated abliteration tool used here (by p-e-w). Finds per-layer directions, searches ablation strengths, and exports a LoRA adapter. (Section 11)

KL divergence

A single number for how different two probability distributions are. Here: how far the edited model's next-token behavior drifted from the original's on benign prompts. 0 = identical; >~0.5 = likely damaged. The damage gauge. (Section 12.5)

Linear representation hypothesis

The well-supported idea that high-level concepts are encoded as linear *directions* in activation space — the reason a single refusal direction exists. (Section 2.3)

llama.cpp

A mature, memory-efficient inference engine that serves GGUF models. Worked where Ollama crashed on this architecture. (Section 13.3)

LoRA

Low-Rank Adaptation. A compact container that stores a weight update as two skinny matrices, `ΔW = (alpha/r)·B·A`. The natural home for abliteration's low-rank edit; ours is rank 3, 69 MB. (Section 10)

Merge

Permanently adding a LoRA's update (`B @ A`) into the base weights to produce one standalone model file. (Sections 10.4, 13.7)

MPOA

This project's name for Heretic's method: directional ablation *parameterized* with a per-layer strength curve and *optimized* by search. (Section 12.1)

NF4

A 4-bit precision used by bitsandbytes. The *editing-time* shrink that let Heretic load the model (~66 GB). Not a serving format. (Section 14.3)

Optuna

The optimization library Heretic uses to search the strength dials, via the TPE sampler. (Section 12.3)

`o_proj` (self_attn.o_proj)

The attention "output projection" — one of the two write-heads per layer; writes the attention result into the residual stream. An abliteration target. (Sections 3.3, 9)

Orthogonal / orthogonalization

Orthogonal = perpendicular (dot product zero, no overlap). *Weight orthogonalization* = editing a write-head so its output is perpendicular to the refusal direction, i.e. contains none of it. (Section 7.3)

Outer product

A column vector times a row vector, producing a matrix. Always rank-1 — the reason removing one direction is a rank-1 (tiny) edit. (Section 10.3)

Pareto front

The set of trade-off options where you cannot improve one objective without worsening another. Here: refusal count vs. KL divergence; you pick a point on it (the elbow). (Section 12.6)

Projection

Rebuilding "how much" (the dot product) into the actual vector component along a direction: `(r_hat · a) r_hat`. Subtracting it removes that direction. (Section 5.3)

Q4_K_M

A llama.cpp 4-bit precision (inside GGUF). The final *serving-time* shrink, ~70 GB. (Section 14.5)

Quantization

Storing weights with fewer bits to shrink the file, at a small accuracy cost — like JPEG quality or audio bitrate. (Section 14.0)

Rank

How many independent "directions of change" an edit contains. Low rank = simple, compact edit. Abliteration is low-rank by construction. (Section 10.1)

Refusal direction (`r_hat`)

The unit-length direction in the residual stream whose presence drives refusals; found by difference-of-means and normalized to length 1. (Sections 5.2, 6.3)

Residual stream

The shared per-token vector that every transformer layer reads from and *adds to* — the model's working memory, and where the whole edit happens. (Section 3.1)

Steering vector

A direction you can push a model's state along to dial a behavior up or down. Refusal behaves like one, which is what makes abliteration possible. (Section 4.3)

Strength (ablation strength)

The per-layer dial controlling how hard the refusal direction is removed: 0 = no change, 1 = full removal, >1 = over-removal. What Heretic searches. (Section 8.3)

Swap

Disk space used as overflow memory, so transient loading spikes page out instead of crashing the machine. (Section 17.5)

Token

A chunk of text (roughly a word or word-fragment) — the unit the model reads and predicts, one at a time. (Section 2.1)

TPE (Tree-structured Parzen Estimator)

The Bayesian-style sampler Optuna used — a "warmer/colder with memory" search that learns which dial settings look promising. (Section 12.3)

Transformer / layer

The model architecture: a tall stack of vector-processing layers, each using attention and MLP machinery to read from and add to the residual stream. 88 layers here. (Sections 3.1–3.2)

Unit vector

A vector rescaled to length 1 — a pure direction with no magnitude. Written with a "hat," e.g. `r_hat`. (Section 5.2)

---

20. Mental checklist

This is your self-test. Cover the answers and try to say each one out loud first — if you can, you have genuinely absorbed the guide, not just read it. The answers are deliberately one-liners; the section pointers tell you where to reread if one is shaky.

Conceptual checklist

- **What is the residual stream?** (Section 3.1)
  - The shared per-token vector that every layer reads from and *adds to* — the model's working memory.
- **Why does abliteration target `o_proj` and `down_proj`, and not the other matrices?** (Section 9)
  - They are the only two matrices per layer that *write* into the residual stream (the "shipping doors"); the others just do internal scratch-work.
- **How is the refusal direction found?** (Section 6.3)
  - Average activations on the refusal-triggering pile minus the average on the harmless pile, then normalize to length 1 (`r_hat`).
- **Why does subtracting two averages isolate refusal — and how can it go wrong?** (Sections 6.4–6.5)
  - Everything the two piles share cancels, leaving the difference (refusal). It goes wrong via *direction leakage* if the piles also differ in some other way (topic, style).
- **What does projection removal actually do?** (Section 5.3)
  - It measures how much refusal an activation contains (`r_hat · a`) and subtracts exactly that much back out: `a − (r_hat · a) r_hat`.
- **What is the difference between runtime hooks and weight orthogonalization?** (Sections 7.2–7.3)
  - Hooks remove refusal *live* every forward pass (reversible, but needs a "censor" present forever); orthogonalization bakes the removal into the weights *once*, giving a standalone model.
- **Why can the abliteration be stored as a tiny LoRA?** (Section 10)
  - Removing one direction is a rank-1 outer product; removing three is rank 3 — naturally low-rank, so it fits in 69 MB.
- **What does the per-layer `strength` dial control, and what happens at the extremes?** (Sections 8.3, 17.1–17.2)
  - How hard refusal is removed at each layer. Too high = over-ablation (damage); too low = under-ablation (refusals remain).
- **What does KL divergence measure here?** (Section 12.5)
  - How far the edited model's next-token probabilities drifted from the original's on benign prompts; the collateral-damage gauge (>~0.5 = likely broken).
- **Why not just pick the trial with zero refusals?** (Sections 12.6–12.7)
  - It can sit at huge KL (Trial 119: KL 5.93) — a broken model. The goal is the best *trade-off* (the elbow), not the fewest refusals.

Project-specific checklist

- **What was the initial refusal rate?**
  - 89/100.
- **Which trial was selected?**
  - Trial 154.
- **What were its metrics?**
  - 6/100 refusals, KL 0.1190.
- **What modules did the adapter target?**
  - `self_attn.o_proj` and `mlp.down_proj`.
- **What was the adapter rank?**
  - Rank 3.
- **Why was BF16 needed?**
  - To get an editable/mergeable full-precision reconstruction from FP8.
- **Why was NF4 needed?**
  - To fit the model for Heretic's direction-finding and optimization run.
- **Why was GGUF Q4_K_M needed?**
  - To produce a final llama.cpp-servable model around 70 GB.

End-to-end summary

The whole project can be compressed into one paragraph:

> Mistral Medium 3.5 128B was too large and awkwardly formatted to edit directly, so the original FP8 weights were dequantized to BF16, then converted to an NF4 bitsandbytes form so Heretic could load the model. Heretic measured refusal directions from harmless vs refusal-triggering prompts, optimized per-layer ablation strengths over 200 trials, and selected Trial 154 because it reduced refusals from 89/100 to 6/100 with KL 0.119. The resulting rank-3 LoRA adapter targeted the two residual-writing modules, `self_attn.o_proj` and `mlp.down_proj`, across 88 layers. The adapter was manually merged into the BF16 base, converted to GGUF, quantized to Q4_K_M, and served with llama.cpp as a roughly 70 GB abliterated model.

Final takeaway

If you have read this far, step back and notice how far the ground has shifted. We started with a single sentence — *find the direction a behavior lives along, and edit the model so it can't go that way* — and you now know what every word of it means, down to the matrix arithmetic and the file formats.

Abliteration is best understood as **geometric model surgery**. It does not teach the model new facts (that is fine-tuning). It does not ask the model nicely (that is prompting). It does not rely on a loophole (that is jailbreaking). It *measures* an internal refusal direction from a few hundred prompts and *edits the two residual-stream write-heads* so that direction can no longer be written. That is the noise-cancelling filter of Section 1, baked into the model's weights.

The genuinely hard parts of this particular project were, surprisingly, **not the math**. The math is a difference of two averages and a projection. The hard parts were logistical — the studio-apartment memory budget, the no-double-quantize wall, five formats, a crash-

prone engine, and a 21-hour dense-model run. That lesson generalizes: with large models, the algorithm is often the easy half; the toolchain is the other half.

The whole method, compressed to four lines:

```
1. Find the refusal direction.           (difference of means -> r_hat)
2. Project it out of the write paths.     (orthogonalize o_proj and down_proj)
3. Choose the least damaging strength.    (Heretic's Pareto elbow, not the extreme)
4. Verify the FINAL served artifact.      (refusals down AND capability intact)
```

And the one number that captures the result: **89/100 refusals became 6/100, at KL 0.119** — almost all of the gate removed, almost none of the brain. That balance — not the absence of refusals, but the *preservation of everything else* — is what separates a successful abliteration from a broken model. Carry that judgement with you; it is the real expertise here.